AP COMPUTER SCIENCE A

Course Framework


elloWorld.ja
*/
public class He
{
pu
ain

Introduction

Computer science involves problem-solving, hardware, and algorithms that help people utilize computers and incorporate multiple perspectives to address real-world problems in contemporary life. As the study of computer science continues to evolve, the careful design of the AP Computer Science A course strives to engage a diverse student population, including female and underrepresented students, by allowing them to discover the power of computer science through rewarding yet challenging concepts.

A well-designed, modern AP Computer Science A course that includes opportunities for students to collaborate to solve problems that interest them, as well as ones that use authentic data sources, can help address traditional issues of equity and access. Such a course can broaden participation in computing while providing a strong and engaging introduction to fundamental areas of the discipline.

The AP Computer Science A course reflects what computer science teachers, professors, and researchers have indicated are the main goals of an introductory, college-level computer science programming course:

- **Design Code**—Determine an appropriate program design and develop algorithms.
- **Develop Code**—Write and implement program code.
- **Analyze Code**—Determine the output or result of given program code or explain why code may not work as intended.
- **Document Code and Computing Systems**—Describe the behavior and conditions that produce the specified results in a program.
- **Use Computers Responsibly**—Understand the ethical and social implications of computer use.

Students practice the computer science skills of designing, developing, and analyzing their own programs to address real-world problems or pursue a passion.

Compatible Curricula

The AP Computer Science A course is compatible with the recommendations of the Association for Computing Machinery (ACM) and the Institute of Electrical and Electronics Engineers Computing Society (IEEE-CS) in the "Software Development Fundamentals" knowledge area, which is recommended for an introductory course. The AP Computer Science A course also integrates topics from the "Programming Languages" and "Algorithms and Complexity" knowledge areas. Teachers can review the **Computer Science Curricula** from ACM and IEEE-CS to see their complete curriculum guidelines.

The AP Computer Science A course vertically aligns with subconcepts in the "Algorithms and Programming" core concept in the **Computer Science Teachers Association (CSTA) K-12 Computer Science Framework**. This vertical alignment allows K-12 CS teachers to make connections from their high school courses to the college-equivalent AP Computer Science A course.

AP Computer Science Program

AP Computer Science A is one of two AP computer science courses available to students. The AP Computer Science Principles course complements AP Computer Science A by teaching the foundational areas of the discipline. Students can take these two courses in either order or concurrently, as allowed by their school.

Resource Requirements

Students should have access to a computer system that represents relatively recent technology. A school should ensure that each student has access to a computer for at least three hours a week; additional time is desirable. Student and instructor access to computers is important during class time, but additional time is essential for students to individually develop solutions to problems.

The computer system must allow students to create, edit, quickly compile, and execute Java programs comparable in size to those found in the AP Computer Science A labs. It is highly desirable that these computers provide students with access to the internet. It is essential that each computer science teacher has internet access.

A school must ensure that each student has a college-level text for individual use both inside and outside of the classroom.

Course Framework Components

Overview

This course framework provides a clear and detailed description of the course requirements necessary for student success. The framework specifies what students should know, be able to do, and understand to qualify for college credit and/or placement.

The course framework includes two essential components:

1 COMPUTATIONAL THINKING PRACTICES

The computational thinking practices are central to the study and practice of computer science. Students should develop and apply the described practices on a regular basis over the span of the course.

2 COURSE CONTENT

The course content is organized into commonly taught units of study that provide a suggested sequence for the course. These units comprise the content and conceptual understandings that colleges and universities typically expect students to be proficient in to qualify for college credit and/or placement.

THIS PAGE IS INTENTIONALLY LEFT BLANK.

1

AP COMPUTER SCIENCE A

Computational Thinking Practices

The computational thinking practices and skills for AP Computer Science A describe what a student should be able to do while exploring course concepts. The table that follows presents these practices along with their corresponding skills that students should develop during the AP Computer Science A course. These skills form the basis of tasks on the AP Computer Science A Exam. While many different skills can be applied to any one content topic, the framework supplies skill focus recommendations for each topic to help assure skill distribution and repetition throughout the course.

The unit guides that follow embed and spiral these practices throughout the course, providing teachers with one way to integrate the practices into the course content, with sufficient repetition to prepare students to apply those skills when taking the AP Computer Science A Exam.

More detailed information about teaching the computational thinking practices and skills can be found in the **Instructional Approaches** section of this publication.

AP COMPUTER SCIENCE A

Computational Thinking Practices

Practice 1	Practice 2	Practice 3	Practice 4	Practice 5
Design Code 1 Determine an appropriate program design and develop algorithms.	***Develop Code*** 2 Write and implement program code.	***Analyze Code*** 3 Determine the output or result of given program code or explain why code may not work as intended.	***Document Code and Computing Systems*** 4 Describe the behavior and conditions that produce specified results in a program.	***Use Computers Responsibly*** 5 Understand the ethical and social implications of computer use.
SKILLS				
1.A Determine an appropriate program design to solve a problem or accomplish a task. 1.B Determine what knowledge can be extracted from data.	2.A Write program code to implement an algorithm. 2.B Write program code involving data abstractions. 2.C Write program code involving procedural abstractions.	3.A Determine the result or output based on statement execution order in an algorithm. 3.B Determine the result or output based on code that contains data abstractions. 3.C Determine the result or output based on code that contains procedural abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.	4.A Describe the behavior of a code segment or program. 4.B Describe the initial conditions that must be met for a code segment to work as intended or described.	5.A Explain how computing impacts society, economy, and culture.

2

AP COMPUTER SCIENCE A

Course Content

The AP Computer Science A course framework provides a clear and detailed description of the course requirements necessary for student success. The framework specifies what students must know, be able to do, and understand with a focus on ideas that encompass core principles, theories, and processes of computer science. The framework also encourages instruction that prepares students for advanced coursework in computer science and its integration into a wide variety of STEM-related fields, as well as for creating useful, reasonable solutions to problems encountered in a rapidly changing world.

UNITS

The course content is organized into commonly taught units. The units have been arranged in a logical sequence frequently found in many college courses and textbooks.

The four units in AP Computer Science A and their relevant weightings on the multiple-choice section of the AP Exam are listed on the following page.

Pacing recommendations on the Course at a Glance page provide suggestions for how to teach the required course content and administer the Progress Checks. The number of suggested class periods is based on a schedule in which the class meets five days a week for 45 minutes each day. While these recommendations have been made to aid planning, teachers should adjust the pacing based on the needs of their students, alternate schedules (e.g., block scheduling), or their school's academic calendar.

Units of Instruction	Approximate Multiple-Choice Exam Weighting
Unit 1: Using Objects and Methods	**15–25%**
Unit 2: Selection and Iteration	**25–35%**
Unit 3: Class Creation	**10–18%**
Unit 4: Data Collections	**30–40%**

Topics

Each unit is divided into teachable segments called topics. Visit the topic pages (starting on page 30) to see all the required content for each topic.

Learning Objectives and Computational Thinking Practices

In AP Computer Science A, every exam question will be aligned to a learning objective and a skill. The learning objectives represent the content domain, while the skill articulates the computational thinking practice required to successfully complete the task. The five categories of computational thinking practices are described as discrete practices; they are, in fact, interrelated and should be applied throughout the course. For a given learning objective, teachers are encouraged to ask the following questions about coding to help students apply the practices:

- How can the concepts of designing a program best be taught to students?
- Does the student's program design maximize its efficiency?
- What programs can be assigned to students that relate to their interests?
- What data can be provided to students to ensure their programs are thoroughly tested?
- What problems could a student solve by writing a computer program?
- What is the best way to teach how to debug programs?
- With all of the data that exists in various databases and websites, how can it be leveraged to help solve problems?
- How could extracting and using data be used to support a claim?

Java Quick Reference

The Java language is extensive, with far more features than could be covered in a single introductory course. The Java Quick Reference is a sheet provided to students during both the multiple-choice and free-response sections of the AP Exam. It includes a list of accessible methods from the Java library that may be included on the exam. The Java Quick Reference sheet can be found in the **Appendix** of this publication, and a copy should be given to students to use throughout the course.

Course at a Glance

Plan

The Course at a Glance provides a useful visual organization of the AP Computer Science A curricular components, including:

- Sequence of units, along with approximate weighting and suggested pacing. Please note, pacing is based on 45-minute class periods, meeting five days each week for a full academic year.
- Progression of topics within each unit.
- Computational thinking practices across units.

Teach

COMPUTATIONAL THINKING PRACTICES

- 1 Design Code
- 2 Develop Code
- 3 Analyze Code
- 4 Document Code and Computing Systems
- 5 Use Computers Responsibly

Required Course Content

Each topic contains required learning objectives and essential knowledge statements that form the basis of the assessment on the AP Exam.

Assess

Assign the Progress Checks—either as homework or in class—for each unit. Each Progress Check contains formative multiple-choice and free-response questions. The feedback from the Progress Checks shows students the areas where they need to focus.

UNIT 1 Using Objects and Methods

~32–34 Class Periods | **15–25%** AP Exam Weighting

1	1.1 Introduction to Algorithms, Programming, and Compilers
1 2	1.2 Variables and Data Types
2 3	1.3 Expressions and Output
2 4	1.4 Assignment Statements and Input
2 4	1.5 Casting and Range of Variables
3	1.6 Compound Assignment Operators
4	1.7 Application Program Interface (API) and Libraries
4	1.8 Documentation with Comments
3	1.9 Method Signatures
2 3	1.10 Calling Class Methods
2 4	1.11 `Math` Class
4	1.12 Objects: Instances of Classes
2	1.13 Object Creation and Storage (Instantiation)
2 3	1.14 Calling Instance Methods
2 3 4	1.15 `String` Manipulation

Progress Check Unit 1 Part 1: Topics 1.1–1.4

Multiple-choice: ~12 questions

Progress Check Unit 1 Part 2: Topics 1.5–1.9

Multiple-choice: ~15 questions

Progress Check Unit 1 Part 3: Topics 1.10–1.15

Multiple-choice: ~18 questions
Free-response: 1 question
- Methods and Control Structures (partial)

UNIT 2 Selection and Iteration

~29–31 Class Periods | **25–35%** AP Exam Weighting

1	2.1 Algorithms with Selection and Repetition
3	2.2 Boolean Expressions
2 3 4	2.3 `if` Statements
2 4	2.4 Nested `if` Statements
2 3	2.5 Compound Boolean Expressions
2 3	2.6 Comparing Boolean Expressions
2 3 4	2.7 `while` Loops
2 3	2.8 `for` Loops
2 3 4	2.9 Implementing Selection and Iteration Algorithms
2 3	2.10 Implementing String Algorithms
2 3 4	2.11 Nested Iteration
4	2.12 Informal Run-Time Analysis

Progress Check Unit 2 Part 1: Topics 2.1–2.6

Multiple-choice: ~18 questions
Free-response: 1 question
- Methods and Control Structures (partial)

Progress Check Unit 2 Part 2: Topics 2.7–2.12

Multiple-choice: ~21 questions
Free-response: 2 questions
- Methods and Control Structures (partial)

NOTE: Partial versions of the free-response questions are provided to prepare students for more complex, full questions that they will encounter on the AP Exam.

UNIT 3 Class Creation

~20–22 Class Periods | **10–18%** AP Exam Weighting

1	**3.1** Abstraction and Program Design
5	**3.2** Impact of Program Design
1 2	**3.3** Anatomy of a Class
2 3	**3.4** Constructors
2 3 4	**3.5** Methods: How to Write Them
2 4	**3.6** Methods: Passing and Returning References of an Object
2 3	**3.7** Class Variables and Methods
3 4	**3.8** Scope and Access
2	**3.9** `this` Keyword

Progress Check Unit 3 Part 1: Topics 3.1–3.4

Multiple-choice: ~12 questions

Progress Check Unit 3 Part 2: Topics 3.5–3.9

Multiple-choice: ~15 questions
Free-response: 2 questions
- Class Design

UNIT 4 Data Collections

~50–52 Class Periods | **30–40%** AP Exam Weighting

1 5	**4.1** Ethical and Social Issues Around Data Collection
1	**4.2** Introduction to Using Data Sets
2 3	**4.3** Array Creation and Access
2 3	**4.4** Array Traversals
2 3 4	**4.5** Implementing Array Algorithms
2 3 4	**4.6** Using Text Files
2	**4.7** Wrapper Classes
2 3	**4.8** `ArrayList` Methods
2 3 4	**4.9** `ArrayList` Traversals
2 3 4	**4.10** Implementing `ArrayList` Algorithms
2 3	**4.11** 2D Array Creation and Access
2 3	**4.12** 2D Array Traversals
2 3 4	**4.13** Implementing 2D Array Algorithms
2 3 4	**4.14** Searching Algorithms
3 4	**4.15** Sorting Algorithms
3 4	**4.16** Recursion
4	**4.17** Recursive Searching and Sorting

Progress Check Unit 4 Part 1: Topics 4.1–4.5

Multiple-choice: ~18 questions
Free-response: 2 questions
- Data Analysis with Array (partial)

Progress Check Unit 4 Part 2: Topics 4.6–4.10

Multiple-choice: ~21 questions
Free-response: 2 questions
- Data Analysis with `ArrayList`

Progress Check Unit 4 Part 3: Topics 4.11–4.17

Multiple-choice: ~21 questions
Free-response: 2 questions
- 2D Array

AP COMPUTER SCIENCE A

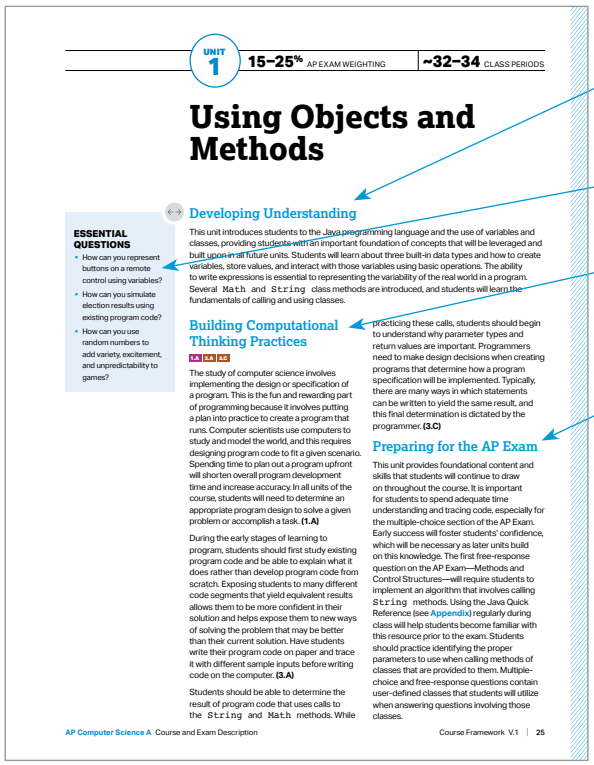
Unit Guides

Introduction

Designed with input from the community of AP Computer Science A educators, the unit guides offer teachers helpful guidance in building students' skills and content knowledge. The suggested sequence was identified through a thorough analysis of the syllabi of highly effective AP teachers and the organization of typical college textbooks.

This unit structure respects new AP teachers' time by providing one possible sequence they can adopt or modify rather than having to build from scratch. An additional benefit is that these units enable the AP Program to provide interested teachers with formative assessments—the Progress Checks—that they can assign their students at the end of each unit to gauge progress toward success on the AP Exam. However, experienced AP teachers who are satisfied with their current course organization and exam results should feel no pressure to adopt these units, which comprise an optional sequence for this course.

Using the Unit Guides



UNIT 1 | 15–25% AP EXAM WEIGHTING | ~32–34 CLASS PERIODS

Using Objects and Methods

ESSENTIAL QUESTIONS

- How can you represent buttons on a remote control using variables?
- How can you simulate election results using existing program code?
- How can you use random numbers to add variety, excitement, and unpredictability to games?

Developing Understanding

This unit introduces students to the Java programming language and the use of variables and classes, providing students with an important foundation of concepts that will be leveraged and built upon in all future units. Students will learn about three built-in data types and how to create variables, store values, and interact with those variables using basic operations. The ability to write expressions is essential to representing the variability of the real world in a program. Several `Math` and `String` class methods are introduced, and students will learn the fundamentals of calling and using classes.

Building Computational Thinking Practices

1.A 3.A 3.C

The study of computer science involves implementing the design or specification of a program. This is the fun and rewarding part of programming because it involves putting a plan into practice to create a program that runs. Computer scientists use computers to study and model the world, and this requires designing program code to fit a given scenario. Spending time to plan out a program upfront will shorten overall program development time and increase accuracy. In all units of the course, students will need to determine an appropriate program design to solve a given problem or accomplish a task. **(1.A)**

During the early stages of learning to program, students should first study existing program code and be able to explain what it does rather than develop program code from scratch. Exposing students to many different code segments that yield equivalent results allows them to be more confident in their solution and helps expose them to new ways of solving the problem that may be better than their current solution. Have students write their program code on paper and trace it with different sample inputs before writing code on the computer. **(3.A)**

Students should be able to determine the result of program code that uses calls to the `String` and `Math` methods. While practicing these calls, students should begin to understand why parameter types and return values are important. Programmers need to make design decisions when creating programs that determine how a program specification will be implemented. Typically, there are many ways in which statements can be written to yield the same result, and this final determination is dictated by the programmer. **(3.C)**

Preparing for the AP Exam

This unit provides foundational content and skills that students will continue to draw on throughout the course. It is important for students to spend adequate time understanding and tracing code, especially for the multiple-choice section of the AP Exam. Early success will foster students' confidence, which will be necessary as later units build on this knowledge. The first free-response question on the AP Exam—Methods and Control Structures—will require students to implement an algorithm that involves calling `String` methods. Using the Java Quick Reference (see Appendix) regularly during class will help students become familiar with this resource prior to the exam. Students should practice identifying the proper parameters to use when calling methods of classes that are provided to them. Multiple-choice and free-response questions contain user-defined classes that students will utilize when answering questions involving those classes.

AP Computer Science A Course and Exam Description — Course Framework V.1 | 25

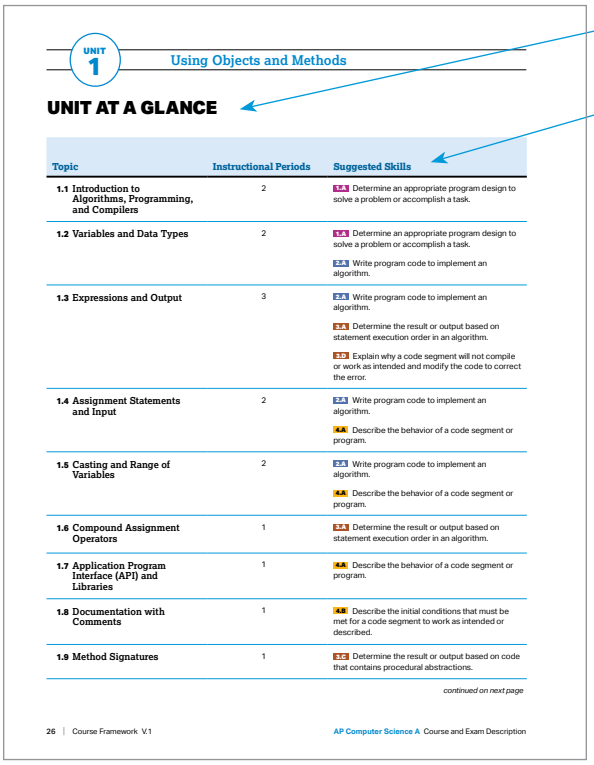
UNIT OPENERS

Developing Understanding provides an overview that contextualizes and situates the key content of the unit within the scope of the course.

The **essential questions** are thought-provoking questions that motivate students and inspire inquiry.

Building Computational Thinking Practices describes specific skills within the practices that are appropriate to focus on in that unit. Certain practices have been noted to indicate areas of emphasis for that unit.

Preparing for the AP Exam provides helpful tips and common student misunderstandings identified from prior exam data.



UNIT 1 | Using Objects and Methods

UNIT AT A GLANCE

Topic	Instructional Periods	Suggested Skills
1.1 Introduction to Algorithms, Programming, and Compilers	2	1.1 Determine an appropriate program design to solve a problem or accomplish a task.
1.2 Variables and Data Types	2	1.1 Determine an appropriate program design to solve a problem or accomplish a task. 2.1 Write program code to implement an algorithm.
1.3 Expressions and Output	3	2.1 Write program code to implement an algorithm. 3.1 Determine the result or output based on statement execution order in an algorithm. 3.3 Explain why a code segment will not compile or work as intended and modify the code to correct the error.
1.4 Assignment Statements and Input	2	2.1 Write program code to implement an algorithm. 4.A Describe the behavior of a code segment or program.
1.5 Casting and Range of Variables	2	2.1 Write program code to implement an algorithm. 4.A Describe the behavior of a code segment or program.
1.6 Compound Assignment Operators	1	3.1 Determine the result or output based on statement execution order in an algorithm.
1.7 Application Program Interface (API) and Libraries	1	4.A Describe the behavior of a code segment or program.
1.8 Documentation with Comments	1	4.B Describe the initial conditions that must be met for a code segment to work as intended or described.
1.9 Method Signatures	1	3.C Determine the result or output based on code that contains procedural abstractions.

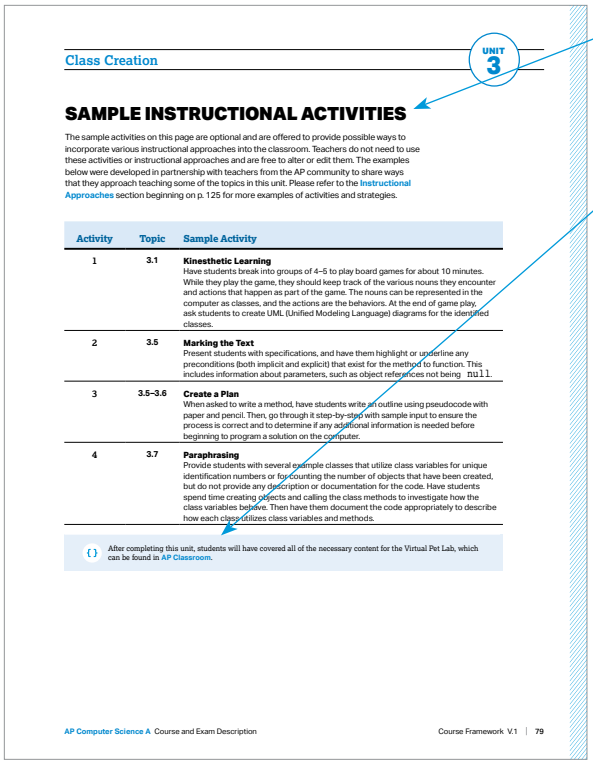
continued on next page

26 | Course Framework V.1 — AP Computer Science A Course and Exam Description

The **Unit at a Glance** table shows the topics and suggested skills.

The **suggested skills** for each topic show possible ways to link the content in that topic to specific AP Computer Science A skills. The individual skills have been thoughtfully chosen in a way that scaffolds the skills throughout the course. However, AP Exam questions can pair the content with any of the skills.

Using the Unit Guides



Class Creation

UNIT 3

SAMPLE INSTRUCTIONAL ACTIVITIES

The sample activities on this page are optional and are offered to provide possible ways to incorporate various instructional approaches into the classroom. Teachers do not need to use these activities or instructional approaches and are free to alter or edit them. The examples below were developed in partnership with teachers from the AP community to share ways that they approach teaching some of the topics in this unit. Please refer to the Instructional Approaches section beginning on p. 125 for more examples of activities and strategies.

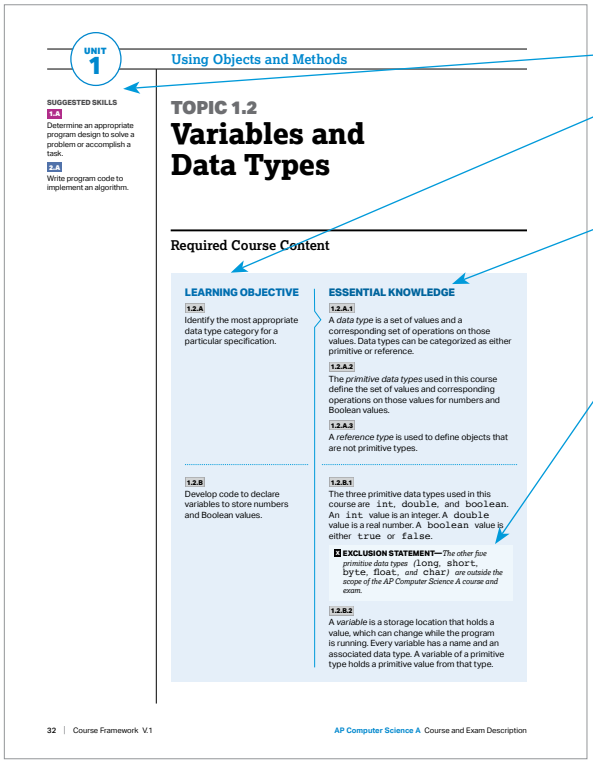
Activity	Topic	Sample Activity
1	3.1	**Kinesthetic Learning** Have students break into groups of 4–5 to play board games for about 10 minutes. While they play the game, they should keep track of the various nouns they encounter and actions that happen as part of the game. The nouns can be represented in the computer as classes, and the actions are the behaviors. At the end of game play, ask students to create UML (Unified Modeling Language) diagrams for the identified classes.
2	3.5	**Marking the Text** Present students with specifications, and have them highlight or underline any preconditions (both implicit and explicit) that exist for the method to function. This includes information about parameters, such as object references not being `null`.
3	3.5–3.6	**Create a Plan** When asked to write a method, have students write an outline using pseudocode with paper and pencil. Then, go through it step-by-step with sample input to ensure the process is correct and to determine if any additional information is needed before beginning to program a solution on the computer.
4	3.7	**Paraphrasing** Provide students with several example classes that utilize class variables for unique identification numbers or for counting the number of objects that have been created, but do not provide any description or documentation for the code. Have students spend time creating objects and calling the class methods to investigate how the class variables behave. Then have them document the code appropriately to describe how each class utilizes class variables and methods.

After completing this unit, students will have covered all of the necessary content for the Virtual Pet Lab, which can be found in AP Classroom.

AP Computer Science A Course and Exam Description — Course Framework V.1 | 79

The **Sample Instructional Activities** includes optional activities that can help teachers tie together the content and skill for a particular topic.

When enough content has been covered to complete an **AP Computer Science A lab**, an icon appears in this section. Teachers can also choose to integrate lab activities earlier as they cover the content.



UNIT 1

Using Objects and Methods

SUGGESTED SKILLS

1.A Determine an appropriate program design to solve a problem or accomplish a task.

2.F Write program code to implement an algorithm.

TOPIC 1.2

Variables and Data Types

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.2.A Identify the most appropriate data type category for a particular specification.	1.2.A.1 A *data type* is a set of values and a corresponding set of operations on those values. Data types can be categorized as either primitive or reference. 1.2.A.2 The *primitive data types* used in this course define the set of values and corresponding operations on those values for numbers and Boolean values. 1.2.A.3 A *reference type* is used to define objects that are not primitive types.
1.2.B Develop code to declare variables to store numbers and Boolean values.	1.2.B.1 The three primitive data types used in this course are `int`, `double`, and `boolean`. An `int` value is an integer. A `double` value is a real number. A `boolean` value is either `true` or `false`. EXCLUSION STATEMENT—*The other five primitive data types* (`long`, `short`, `byte`, `float`, *and* `char`) *are outside the scope of the AP Computer Science A course and exam.* 1.2.B.2 A *variable* is a storage location that holds a value, which can change while the program is running. Every variable has a name and an associated data type. A variable of a primitive type holds a primitive value from that type.

32 | Course Framework V.1 — AP Computer Science A Course and Exam Description

TOPIC PAGES

The **suggested skills** offer possible skills to pair with the topic.

Learning objectives define what a student needs to be able to do with content knowledge in order to progress through the course.

Essential knowledge statements define the required content knowledge associated with each learning objective assessed on the AP Exam.

Exclusion statements provide guidance to teachers regarding the content exclusions of the AP Computer Science A course. The content in the exclusion statements will not be assessed on the AP Computer Science A Exam. However, such content may be provided as background or additional information for the concepts and skills being assessed in lab activities.

THIS PAGE IS INTENTIONALLY LEFT BLANK.

AP COMPUTER SCIENCE A

UNIT 1

Using Objects and Methods

15–25%
AP EXAM WEIGHTING

~32–34
CLASS PERIODS

Remember to go to **AP Classroom** to assign students the online **Progress Checks** for this unit.

Whether assigned as homework or completed in class, the **Progress Checks** provide each student with immediate feedback related to this unit's topics and skills.

Progress Check Unit 1 Part 1: Topics 1.1–1.4

Multiple-choice: ~12 questions

Progress Check Unit 1 Part 2: Topics 1.5–1.9

Multiple-choice: ~15 questions

Progress Check Unit 1 Part 3: Topics 1.10–1.15

Multiple-choice: ~18 questions
Free-response: 1 question

- Methods and Control Structures (partial)

UNIT 1 | 15–25% AP EXAM WEIGHTING | ~32–34 CLASS PERIODS

Using Objects and Methods

ESSENTIAL QUESTIONS

- How can you represent buttons on a remote control using variables?
- How can you simulate election results using existing program code?
- How can you use random numbers to add variety, excitement, and unpredictability to games?

Developing Understanding

This unit introduces students to the Java programming language and the use of variables and classes, providing students with an important foundation of concepts that will be leveraged and built upon in all future units. Students will learn about three built-in data types and how to create variables, store values, and interact with those variables using basic operations. The ability to write expressions is essential to representing the variability of the real world in a program. Several `Math` and `String` class methods are introduced, and students will learn the fundamentals of calling and using classes.

Building Computational Thinking Practices

1.A 3.A 3.C

The study of computer science involves implementing the design or specification of a program. This is the fun and rewarding part of programming because it involves putting a plan into practice to create a program that runs. Computer scientists use computers to study and model the world, and this requires designing program code to fit a given scenario. Spending time to plan out a program upfront will shorten overall program development time and increase accuracy. In all units of the course, students will need to determine an appropriate program design to solve a given problem or accomplish a task. **(1.A)**

During the early stages of learning to program, students should first study existing program code and be able to explain what it does rather than develop program code from scratch. Exposing students to many different code segments that yield equivalent results allows them to be more confident in their solution and helps expose them to new ways of solving the problem that may be better than their current solution. Have students write their program code on paper and trace it with different sample inputs before writing code on the computer. **(3.A)**

Students should be able to determine the result of program code that uses calls to the `String` and `Math` methods. While practicing these calls, students should begin to understand why parameter types and return values are important. Programmers need to make design decisions when creating programs that determine how a program specification will be implemented. Typically, there are many ways in which statements can be written to yield the same result, and this final determination is dictated by the programmer. **(3.C)**

Preparing for the AP Exam

This unit provides foundational content and skills that students will continue to draw on throughout the course. It is important for students to spend adequate time understanding and tracing code, especially for the multiple-choice section of the AP Exam. Early success will foster students' confidence, which will be necessary as later units build on this knowledge. The first free-response question on the AP Exam—Methods and Control Structures—will require students to implement an algorithm that involves calling `String` methods. Using the Java Quick Reference (see **Appendix**) regularly during class will help students become familiar with this resource prior to the exam. Students should practice identifying the proper parameters to use when calling methods of classes that are provided to them. Multiple-choice and free-response questions contain user-defined classes that students will utilize when answering questions involving those classes.

UNIT AT A GLANCE

Topic	Instructional Periods	Suggested Skills
1.1 Introduction to Algorithms, Programming, and Compilers	2	1.A Determine an appropriate program design to solve a problem or accomplish a task.
1.2 Variables and Data Types	2	1.A Determine an appropriate program design to solve a problem or accomplish a task. 2.A Write program code to implement an algorithm.
1.3 Expressions and Output	3	2.A Write program code to implement an algorithm. 3.A Determine the result or output based on statement execution order in an algorithm. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.
1.4 Assignment Statements and Input	2	2.A Write program code to implement an algorithm. 4.A Describe the behavior of a code segment or program.
1.5 Casting and Range of Variables	2	2.A Write program code to implement an algorithm. 4.A Describe the behavior of a code segment or program.
1.6 Compound Assignment Operators	1	3.A Determine the result or output based on statement execution order in an algorithm.
1.7 Application Program Interface (API) and Libraries	1	4.A Describe the behavior of a code segment or program.
1.8 Documentation with Comments	1	4.B Describe the initial conditions that must be met for a code segment to work as intended or described.
1.9 Method Signatures	1	3.C Determine the result or output based on code that contains procedural abstractions.

continued on next page

UNIT AT A GLANCE *(cont'd)*

Topic	Instructional Periods	Suggested Skills
1.10 Calling Class Methods	3	2.C Write program code involving procedural abstractions. 3.C Determine the result or output based on code that contains procedural abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.
1.11 `Math` Class	2	2.C Write program code involving procedural abstractions. 4.A Describe the behavior of a code segment or program.
1.12 Objects: Instances of Classes	1	4.A Describe the behavior of a code segment or program.
1.13 Object Creation and Storage (Instantiation)	2	2.B Write program code involving data abstractions.
1.14 Calling Instance Methods	2	2.C Write program code involving procedural abstractions. 3.C Determine the result or output based on code that contains procedural abstractions.
1.15 `String` Manipulation	4	2.C Write program code involving procedural abstractions. 3.C Determine the result or output based on code that contains procedural abstractions. 4.A Describe the behavior of a code segment or program. 4.B Describe the initial conditions that must be met for a code segment to work as intended or described.

Go to **AP Classroom** to assign the **Progress Checks** for Unit 1.
Review the results in class to identify and address any student misunderstandings.

SAMPLE INSTRUCTIONAL ACTIVITIES

The sample activities on this page are optional and are offered to provide possible ways to incorporate various instructional approaches into the classroom. Teachers do not need to use these activities or instructional approaches and are free to alter or edit them. The examples below were developed in partnership with teachers from the AP community to share ways that they approach teaching some of the topics in this unit. Please refer to the **Instructional Approaches** section beginning on p. 125 for more examples of activities and strategies.

Activity	Topic	Sample Activity
1	1.3	**Activating Prior Knowledge** The basic arithmetic operators of +, −, /, and * are similar to what students have experienced in math class or when using a calculator. Give students a list of expressions and ask them to apply what they know from math class to evaluate the meaning of the expressions. Have them verify their results by writing a small program.
2	1.5	**Predict and Confirm** Provide students with several statements that involve casting. Each cast should be on a different value in the statement. Have students predict the resulting value. For any statements that would not compile or work as intended, have students explain the problem and propose a solution. They should verify their results by putting those results into a compiler.
3	1.6	**Error Analysis** Provide students with code that contains syntax errors. Ask students to identify and correct the errors in the provided code. Then have them verify their conclusion by using a compiler and an integrated development environment (IDE) that does not autocorrect errors.
4	1.6	**Sharing and Responding** Put students into groups of two. Provide each student with a different set of statements; in each pair, one student should have a list of statements that contain compound assignment operators, while the other student has a list of statements that accomplish the same thing without using compound statements. Be sure the statements are in a different order. Students should take turns describing what a statement does to their partner, and the partner should determine which statement of theirs is equivalent to the one being described.
5	1.13	**Using Manipulatives** When introducing students to the idea of creating objects, use a cookie cutter and modeling clay or dough, with the cutter representing the class and the cut dough representing the objects. For each object cut, write the instantiation. Ask students to describe what the code is doing and how the different parameter values (e.g., thickness, color) change the object that was created.
6	1.13	**Marking the Text** Provide students with several statements that define a variable and create an object on a single line. Have students mark up the statements by circling the assignment operator and the `new` keyword. Then, have students underline the variable type and the constructor. Lastly, have them draw a rectangle around the list of actual parameters being passed to the constructor. Using these marked-up statements, ask students to create several new variables and objects.

Activity	Topic	Sample Activity
7	1.15	**Think-Pair-Share** Provide students with several code segments, each with a missing expression that would contain a call to a method in the `String` class and a description of the intended outcome of each code segment. Ask students which statement should be used to complete the code segment. Have them share their responses with a partner to compare answers and come to an agreement, and then have groups share with the entire class.

{ } After completing this unit, students will have covered all of the necessary content for the Receipt Lab, which can be found in **AP Classroom**.

SUGGESTED SKILLS

1.A

Determine an appropriate program design to solve a problem or accomplish a task.

TOPIC 1.1
Introduction to Algorithms, Programming, and Compilers

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.1.A Represent patterns and algorithms found in everyday life using written language or diagrams.	**1.1.A.1** *Algorithms* define step-by-step processes to follow when completing a task or solving a problem. These algorithms can be represented using written language or diagrams. **1.1.A.2** *Sequencing* defines an order for when steps in a process are completed. Steps in a process are completed one at a time.
1.1.B Explain the code compilation and execution process.	**1.1.B.1** Code can be written in any text editor; however, an *integrated development environment (IDE)* is often used to write programs because it provides tools for a programmer to write, compile, and run code. **1.1.B.2** A *compiler* checks code for some errors. Errors detectable by the compiler need to be fixed before the program can be run.
1.1.C Identify types of programming errors.	**1.1.C.1** A *syntax error* is a mistake in the program where the rules of the programming language are not followed. These errors are detected by the compiler.

continued on next page

LEARNING OBJECTIVE

1.1.C

Identify types of programming errors.

ESSENTIAL KNOWLEDGE

1.1.C.2

A *logic error* is a mistake in the algorithm or program that causes it to behave incorrectly or unexpectedly. These errors are detected by testing the program with specific data to see if it produces the expected outcome.

1.1.C.3

A *run-time error* is a mistake in the program that occurs during the execution of a program. Run-time errors typically cause the program to terminate abnormally.

1.1.C.4

An *exception* is a type of run-time error that occurs as a result of an unexpected error that was not detected by the compiler. It interrupts the normal flow of the program's execution.

SUGGESTED SKILLS

1.A
Determine an appropriate program design to solve a problem or accomplish a task.

2.A
Write program code to implement an algorithm.

TOPIC 1.2
Variables and Data Types

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.2.A Identify the most appropriate data type category for a particular specification.	**1.2.A.1** A *data type* is a set of values and a corresponding set of operations on those values. Data types can be categorized as either primitive or reference. **1.2.A.2** The *primitive data types* used in this course define the set of values and corresponding operations on those values for numbers and Boolean values. **1.2.A.3** A *reference type* is used to define objects that are not primitive types.
1.2.B Develop code to declare variables to store numbers and Boolean values.	**1.2.B.1** The three primitive data types used in this course are `int`, `double`, and `boolean`. An `int` value is an integer. A `double` value is a real number. A `boolean` value is either `true` or `false`. **EXCLUSION STATEMENT**—*The other five primitive data types (`long`, `short`, `byte`, `float`, and `char`) are outside the scope of the AP Computer Science A course and exam.* **1.2.B.2** A *variable* is a storage location that holds a value, which can change while the program is running. Every variable has a name and an associated data type. A variable of a primitive type holds a primitive value from that type.

TOPIC 1.3
Expressions and Output

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

3.A
Determine the result or output based on statement execution order in an algorithm.

3.D
Explain why a code segment will not compile or work as intended and modify the code to correct the error.

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.3.A Develop code to generate output and determine the result that would be displayed.	**1.3.A.1** `System.out.print` and `System.out.println` display information on the computer display. `System.out.println` moves the cursor to a new line after the information has been displayed, while `System.out.print` does not.
1.3.B Develop code to utilize string literals and determine the result of using string literals.	**1.3.B.1** A *literal* is the code representation of a fixed value. **1.3.B.2** A *string literal* is a sequence of characters enclosed in double quotes. **1.3.B.3** *Escape sequences* are special sequences of characters that can be included in a string. They start with a `\` and have a special meaning in Java. Escape sequences used in this course include double quote `\"`, backslash `\\`, and newline `\n`.
1.3.C Develop code for arithmetic expressions and determine the result of these expressions.	**1.3.C.1** *Arithmetic expressions*, which consist of numeric values, variables, and operators, include expressions of type `int` and `double`.

continued on next page

LEARNING OBJECTIVE

1.3.C

Develop code for arithmetic expressions and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

1.3.C.2

The *arithmetic operators* consist of addition `+`, subtraction `-`, multiplication `*`, division `/`, and remainder `%`. An arithmetic operation that uses two `int` values will evaluate to an `int` value. An arithmetic operation that uses at least one `double` value will evaluate to a `double` value.

EXCLUSION STATEMENT—*Expressions that result in special double values (e.g., infinities and NaN) are outside the scope of the AP Computer Science A course and exam.*

1.3.C.3

When dividing numeric values that are both `int` values, the result is only the integer portion of the quotient. When dividing numeric values that use at least one `double` value, the result is the quotient.

1.3.C.4

The remainder operator `%` is used to compute the remainder when one number `a` is divided by another number `b`.

EXCLUSION STATEMENT—*The use of values less than* `0` *for* `a` *and the use of values less than or equal to* `0` *for* `b` *is outside the scope of the AP Computer Science A course and exam.*

1.3.C.5

Operators can be used to construct compound expressions. At compile time, numeric values are associated with operators according to operator precedence to determine how they are grouped. Parentheses can be used to modify operator precedence. Multiplication, division, and remainder have precedence over addition and subtraction. Operators with the same precedence are evaluated from left to right.

1.3.C.6

An attempt to divide an integer by the integer zero will result in an `ArithmeticException`.

EXCLUSION STATEMENT—*The use of dividing by zero when one numeric value is a* `double` *is outside the scope of the AP Computer Science A course and exam.*

TOPIC 1.4
Assignment Statements and Input

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

4.A
Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.4.A Develop code for assignment statements with expressions and determine the value that is stored in the variable as a result of these statements.	**1.4.A.1** Every variable must be assigned a value before it can be used in an expression. That value must be from a compatible data type. A variable is initialized the first time it is assigned a value. Reference types can be assigned a new object or `null` if there is no object. The literal `null` is a special value used to indicate that a reference is not associated with any object. **1.4.A.2** The assignment operator `=` allows a program to initialize or change the value stored in a variable. The value of the expression on the right is stored in the variable on the left. **EXCLUSION STATEMENT**—*The use of assignment operators inside expressions (e.g.,* `a = b = 4;` *or* `a[i += 5]`*) is outside the scope of the AP Computer Science A course and exam.* **1.4.A.3** During execution, an expression is evaluated to produce a single value. The value of an expression has a type based on the evaluation of the expression.
1.4.B Develop code to read input.	**1.4.B.1** Input can come in a variety of forms, such as tactile, audio, visual, or text. The `Scanner` class is one way to obtain text input from the keyboard. **EXCLUSION STATEMENT**—*Any specific form of input from the user is outside the scope of the AP Computer Science A course and exam.*

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

4.A
Describe the behavior of a code segment or program.

TOPIC 1.5
Casting and Range of Variables

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.5.A Develop code to cast primitive values to different primitive types in arithmetic expressions and determine the value that is produced as a result.	**1.5.A.1** The *casting operators* `(int)` and `(double)` can be used to convert from a `double` value to an `int` value (or vice versa). **1.5.A.2** Casting a `double` value to an `int` value causes the digits to the right of the decimal point to be truncated. **1.5.A.3** Some code causes `int` values to be automatically cast (widened) to `double` values. **1.5.A.4** Values of type `double` can be rounded to the nearest integer by `(int)(x + 0.5)` for non-negative numbers or `(int)(x - 0.5)` for negative numbers.
1.5.B Describe conditions when an integer expression evaluates to a value out of range.	**1.5.B.1** The constant `Integer.MAX_VALUE` holds the value of the largest possible `int` value. The constant `Integer.MIN_VALUE` holds the value of the smallest possible `int` value. **1.5.B.2** Integer values in Java are represented by values of type `int`, which are stored using a finite amount (4 bytes) of memory. Therefore, an `int` value must be in the range from `Integer.MIN_VALUE` to `Integer.MAX_VALUE` inclusive.

continued on next page

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.5.B Describe conditions when an integer expression evaluates to a value out of range.	1.5.B.3 If an expression would evaluate to an `int` value outside of the allowed range, an integer overflow occurs. The result is an `int` value in the allowed range but not necessarily the value expected.
1.5.C Describe conditions that limit accuracy of expressions.	1.5.C.1 Computers allot a specified amount of memory to store data based on the data type. If an expression would evaluate to a `double` that is more precise than can be stored in the allotted amount of memory, a round-off error occurs. The result will be rounded to the representable value. To avoid rounding errors that naturally occur, use `int` values. **EXCLUSION STATEMENT—***Other special decimal data types that can be used to avoid rounding errors are outside the scope of the AP Computer Science A course and exam.*

SUGGESTED SKILLS

3.A

Determine the result or output based on statement execution order in an algorithm.

TOPIC 1.6
Compound Assignment Operators

Required Course Content

LEARNING OBJECTIVE

1.6.A

Develop code for assignment statements with compound assignment operators and determine the value that is stored in the variable as a result.

ESSENTIAL KNOWLEDGE

1.6.A.1

Compound assignment operators `+=`, `-=`, `*=`, `/=`, and `%=` can be used in place of the assignment operator in numeric expressions. A compound assignment operator performs the indicated arithmetic operation between the value on the left and the value on the right and then assigns the result to the variable on the left.

1.6.A.2

The post-increment operator `++` and post-decrement operator `--` are used to add `1` or subtract `1` from the stored value of a numeric variable. The new value is assigned to the variable.

EXCLUSION STATEMENT—*The use of increment and decrement operators in prefix form (e.g.,* `++x`*) is outside the scope of the AP Computer Science A course and exam. The use of increment and decrement operators inside other expressions (e.g.,* `arr[x++]`*) is outside the scope of the AP Computer Science A course and exam.*

TOPIC 1.7
Application Program Interface (API) and Libraries

Required Course Content

LEARNING OBJECTIVE

1.7.A
Identify the attributes and behaviors of a class found in the libraries contained in an API.

ESSENTIAL KNOWLEDGE

1.7.A.1
Libraries are collections of classes. An *application programming interface (API)* specification informs the programmer how to use those classes. Documentation found in API specifications and libraries is essential to understanding the attributes and behaviors of a class defined by the API. A *class* defines a specific reference type. Classes in the APIs and libraries are grouped into packages. Existing classes and class libraries can be utilized to create objects.

1.7.A.2
Attributes refer to the data related to the class and are stored in variables. *Behaviors* refer to what instances of the class can do (or what can be done with them) and are defined by methods.

SUGGESTED SKILLS

4.A
Describe the behavior of a code segment or program.

SUGGESTED SKILLS

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

TOPIC 1.8
Documentation with Comments

Required Course Content

LEARNING OBJECTIVE

1.8.A

Describe the functionality and use of code through comments.

ESSENTIAL KNOWLEDGE

1.8.A.1

Comments are written for both the original programmer and other programmers to understand the code and its functionality, but are ignored by the compiler and are not executed when the program is run. Three types of comments in Java include `/* */`, which generates a block of comments; `//`, which generates a comment on one line; and `/** */`, which are Javadoc comments and are used to create API documentation.

1.8.A.2

A *precondition* is a condition that must be true just prior to the execution of a method in order for it to behave as expected. There is no expectation that the method will check to ensure preconditions are satisfied.

1.8.A.3

A *postcondition* is a condition that must always be true after the execution of a method. Postconditions describe the outcome of the execution in terms of what is being returned or the current value of the attributes of an object.

TOPIC 1.9
Method Signatures

SUGGESTED SKILLS

3.C

Determine the result or output based on code that contains procedural abstractions.

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.9.A Identify the correct method to call based on documentation and method signatures.	1.9.A.1 A *method* is a named block of code that only runs when it is called. A *block of code* is any section of code that is enclosed in braces. *Procedural abstraction* allows a programmer to use a method by knowing what the method does even if they do not know how the method was written. 1.9.A.2 A *parameter* is a variable declared in the header of a method or constructor and can be used inside the body of the method. This allows values or arguments to be passed and used by a method or constructor. A *method signature* for a method with parameters consists of the method name and the ordered list of parameter types. A method signature for a method without parameters consists of the method name and an empty parameter list.
1.9.B Describe how to call methods.	1.9.B.1 A *void method* does not have a return value and is therefore not called as part of an expression. 1.9.B.2 A *non-void method* returns a value that is the same type as the return type in the header. To use the return value when calling a non-void method, it must be stored in a variable or used as part of an expression.

continued on next page

LEARNING OBJECTIVE

1.9.B

Describe how to call methods

ESSENTIAL KNOWLEDGE

1.9.B.3

An *argument* is a value that is passed into a method when the method is called. The arguments passed to a method must be compatible in number and order with the types identified in the parameter list of the method signature. When calling methods, arguments are passed using call by value. *Call by value* initializes the parameters with copies of the arguments.

1.9.B.4

Methods are said to be *overloaded* when there are multiple methods with the same name but different signatures.

1.9.B.5

A *method call* interrupts the sequential execution of statements, causing the program to first execute the statements in the method before continuing. Once the last statement in the method has been executed or a return statement is executed, the flow of control is returned to the point immediately following where the method was called.

TOPIC 1.10
Calling Class Methods

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.10.A Develop code to call class methods and determine the result of those calls.	**1.10.A.1** *Class methods* are associated with the class, not instances of the class. Class methods include the keyword `static` in the header before the method name.
	1.10.A.2 Class methods are typically called using the class name along with the dot operator. When the method call occurs in the defining class, the use of the class name is optional in the call.

SUGGESTED SKILLS

2.C Write program code involving procedural abstractions.

3.C Determine the result or output based on code that contains procedural abstractions.

3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.

SUGGESTED SKILLS

2.C
Write program code involving procedural abstractions.

4.A
Describe the behavior of a code segment or program.

TOPIC 1.11
Math Class

Required Course Content

LEARNING OBJECTIVE

1.11.A
Develop code to write expressions that incorporate calls to built-in mathematical libraries and determine the value that is produced as a result.

ESSENTIAL KNOWLEDGE

1.11.A.1
The `Math` class is part of the `java.lang` package. Classes in the `java.lang` package are available by default.

1.11.A.2
The `Math` class contains only class methods. The following `Math` class methods—including what they do and when they are used—are part of the Java Quick Reference:

- `static int abs(int x)` returns the absolute value of an `int` value.
- `static double abs(double x)` returns the absolute value of a `double` value.
- `static double pow(double base, double exponent)` returns the value of the first parameter raised to the power of the second parameter.
- `static double sqrt(double x)` returns the nonnegative square root of a `double` value.
- `static double random()` returns a `double` value greater than or equal to `0.0` and less than `1.0`.

1.11.A.3
The values returned from `Math.random()` can be manipulated using arithmetic and casting operators to produce a random `int` or `double` in a defined range based on specified criteria. Each endpoint of the range can be *inclusive*, meaning the value is included, or *exclusive*, meaning the value is not included.

TOPIC 1.12
Objects: Instances of Classes

SUGGESTED SKILLS

4.A

Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.12.A Explain the relationship between a class and an object.	**1.12.A.1** An *object* is a specific instance of a class with defined attributes. A *class* is the formal implementation, or blueprint, of the attributes and behaviors of an object.
	1.12.A.2 A class hierarchy can be developed by putting common attributes and behaviors of related classes into a single class called a *superclass*. Classes that extend a superclass, called *subclasses*, can draw upon the existing attributes and behaviors of the superclass without replacing these in the code. This creates an *inheritance relationship* from the subclasses to the superclass.
	EXCLUSION STATEMENT—*Designing and implementing inheritance relationships are outside the scope of the AP Computer Science A course and exam.*
	1.12.A.3 All classes in Java are subclasses of the `Object` class.
1.12.B Develop code to declare variables to store reference types.	**1.12.B.1** A variable of a reference type holds an object reference, which can be thought of as the memory address of that object.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

TOPIC 1.13
Object Creation and Storage (Instantiation)

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.13.A Identify, using its signature, the correct constructor being called.	**1.13.A.1** A class contains *constructors* that are called to create objects. They have the same name as the class. **1.13.A.2** A *constructor signature* consists of the constructor's name, which is the same as the class name, and the ordered list of parameter types. The parameter list, in the header of a constructor, lists the types of the values that are passed and their variable names. **1.13.A.3** Constructors are said to be overloaded when there are multiple constructors with different signatures.
1.13.B Develop code to declare variables of the correct types to hold object references.	**1.13.B.1** A variable of a reference type holds an object reference or, if there is no object, `null`.
1.13.C Develop code to create an object by calling a constructor.	**1.13.C.1** An object is typically created using the keyword `new` followed by a call to one of the class's constructors. **1.13.C.2** Parameters allow constructors to accept values to establish the initial values of the attributes of the object.

continued on next page

LEARNING OBJECTIVE

1.13.C

Develop code to create an object by calling a constructor.

ESSENTIAL KNOWLEDGE

1.13.C.3

A *constructor argument* is a value that is passed into a constructor when the constructor is called. The arguments passed to a constructor must be compatible in order and number with the types identified in the parameter list in the constructor signature. When calling constructors, arguments are passed using call by value. Call by value initializes the parameters with copies of the arguments.

1.13.C.4

A constructor call interrupts the sequential execution of statements, causing the program to first execute the statements in the constructor before continuing. Once the last statement in the constructor has been executed, the flow of control is returned to the point immediately following where the constructor was called.

SUGGESTED SKILLS

2.C
Write program code involving procedural abstractions.

3.C
Determine the result or output based on code that contains procedural abstractions.

TOPIC 1.14
Calling Instance Methods

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.14.A Develop code to call instance methods and determine the result of these calls.	**1.14.A.1** *Instance methods* are called on objects of the class. The dot operator is used along with the object name to call instance methods.
	1.14.A.2 A method call on a `null` reference will result in a `NullPointerException`.

TOPIC 1.15
`String` Manipulation

Required Course Content

LEARNING OBJECTIVE

1.15.A

Develop code to create string objects and determine the result of creating and combining strings.

ESSENTIAL KNOWLEDGE

1.15.A.1

A `String` object represents a sequence of characters and can be created by using a string literal or by calling the `String` class constructor.

1.15.A.2

The `String` class is part of the `java.lang` package. Classes in the `java.lang` package are available by default.

1.15.A.3

A `String` object is immutable, meaning once a `String` object is created, its attributes cannot be changed. Methods called on a `String` object do not change the content of the `String` object.

1.15.A.4

Two `String` objects can be concatenated together or combined using the `+` or `+=` operator, resulting in a new `String` object. A primitive value can be concatenated with a `String` object. This causes the implicit conversion of the primitive value to a `String` object.

continued on next page

SUGGESTED SKILLS

2.C
Write program code involving procedural abstractions.

3.C
Determine the result or output based on code that contains procedural abstractions.

4.A
Describe the behavior of a code segment or program.

4.B
Describe the initial conditions that must be met for a code segment to work as intended or described.

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
1.15.A Develop code to create string objects and determine the result of creating and combining strings.	1.15.A.5 A `String` object can be concatenated with any object, which implicitly calls the object's `toString` method (a behavior that is guaranteed to exist by the inheritance relationship every class has with the `Object` class). An object's `toString` method returns a string value representing the object. Subclasses of `Object` often override the `toString` method with class-specific implementation. *Method overriding* occurs when a public method in a subclass has the same method signature as a public method in the superclass, but the behavior of the method is specific to the subclass. **EXCLUSION STATEMENT—***Overriding the* `toString` *method of a class is outside the scope of the AP Computer Science A course and exam.*
1.15.B Develop code to call methods on string objects and determine the result of calling these methods.	1.15.B.1 A `String` object has index values from 0 to one less than the length of the string. Attempting to access indices outside this range will result in a `StringIndexOutOfBoundsException`.

continued on next page

LEARNING OBJECTIVE

1.15.B

Develop code to call methods on string objects and determine the result of calling these methods.

ESSENTIAL KNOWLEDGE

1.15.B.2

The following `String` methods—including what they do and when they are used—are part of the Java Quick Reference:

- `int length()` returns the number of characters in a `String` object.
- `String substring(int from, int to)` returns the substring beginning at index `from` and ending at index `to - 1`.
- `String substring(int from)` returns `substring(from, length())`.
- `int indexOf(String str)` returns the index of the first occurrence of `str`; returns `-1` if not found.
- `boolean equals(Object other)` returns `true` if `this` corresponds to the same sequence of characters as `other`; returns `false` otherwise.
- `int compareTo(String other)` returns a value < 0 if `this` is less than `other`; returns zero if `this` is equal to `other`; returns a value > 0 if `this` is greater than `other`. Strings are ordered based upon the alphabet.

X EXCLUSION STATEMENT—*Using the* `equals` *method to compare one* `String` *object with an object of a type other than* `String` *is outside the scope of the AP Computer Science A course and exam.*

1.15.B.3

A string identical to the single element substring at position `index` can be created by calling `substring(index, index + 1)`.

THIS PAGE IS INTENTIONALLY LEFT BLANK.

AP COMPUTER SCIENCE A

UNIT 2

Selection and Iteration

25–35%
AP EXAM WEIGHTING

~29–31
CLASS PERIODS

Remember to go to **AP Classroom** to assign students the online **Progress Checks** for this unit.

Whether assigned as homework or completed in class, the **Progress Checks** provide each student with immediate feedback related to this unit's topics and skills.

Progress Check Unit 2 Part 1: Topics 2.1–2.6

Multiple-choice: ~18 questions
Free-response: 1 question

- Methods and Control Structures (partial)

Progress Check Unit 2 Part 2: Topics 2.7–2.12

Multiple-choice: ~21 questions
Free-response: 2 question

- Methods and Control Structures (partial)

UNIT 2

25–35% AP EXAM WEIGHTING | **~29–31** CLASS PERIODS

Selection and Iteration

ESSENTIAL QUESTIONS

- Why is selection a necessary part of programming languages?
- How can you use different conditional statements to write a pick-your-own-path interactive story?
- How does iteration improve programs and reduce the amount of program code necessary to complete a task?
- Why are standard algorithms useful when solving new problems?

Developing Understanding

Algorithms are composed of three building blocks: sequencing, selection, and iteration. While Unit 1 introduced sequencing, this unit focuses on selection and iteration. Selection is important to a computer program because it gives the programmer the ability to make a decision and respond to that decision using conditional statements. These allow programmers to incorporate choice into their programs: to create games that react to user interactions, to develop simulations that are more real world by allowing for variability, or to discover new knowledge in a sea of information by filtering out irrelevant data. Iteration is a form of repetition and changes the flow of control by repeating a segment of code. It is represented by `while` and `for` loops. In addition, students will build on the introduction of Boolean variables in Unit 1 by writing Boolean expressions with relational and logical operators. This unit also introduces several standard algorithms that use iteration. Knowledge of standard algorithms makes solving similar problems easier, as algorithms can be modified or combined to suit new situations.

Building Computational Thinking Practices

2.A 3.D 4.A

Selection and iteration allow programmers to incorporate choices and repetition into their programs. Understanding program code that uses those processes allows students to write programs that solve a wide variety of problems. In addition to developing their own programs, students should practice completing partially written program code to fulfill a specification. This builds their confidence and provides them opportunities to be successful during the early stages of learning. **(2.A)**

When writing code, errors are inevitable. To learn how to identify and correct errors, students need exposure to the error messages generated by the compiler. Sometimes students struggle to explain why a code segment will not compile or work as intended. It helps to have students work with a collaborative partner to practice verbally explaining the issues. The ability to describe why a code segment does not work correctly assists students in the development of solutions. **(3.D)**

Students should also be able to describe the behavior of a code segment or program. Have students write program code on paper and trace that code with different sample inputs. Spending time analyzing existing program code provides opportunities for students to better understand how to solve their own problems and the implications associated with code changes. **(4.A)**

Preparing for the AP Exam

The study of computer science involves the analysis of program code. On the multiple-choice section of the AP Exam, students will be asked to determine the result of a given program code segment based on specific input and on the behavior of program code in general or

to complete missing code in a described task. Students should know how to trace program code when testing programs to ensure that all conditions are met. Testing for the different expected behaviors of conditional and iterative statements is a critical part of program development and is useful when writing program code or analyzing given code segments. Students often struggle in situations that warrant variation in the Boolean condition of loops, such as when they want to terminate a loop early. Early termination of a loop requires the use of conditional statements within the body of the loop. If the order of the program statements is incorrect, the early termination may be triggered too early or not at all. Provide students with practice ordering statements by giving them strips of paper, each with a line of program code that can be used to assemble the correct and incorrect solutions. Ask them to reassemble the code and trace it to see if it is correct.

Using manipulatives in this way makes it easier for students to rearrange the order of the program code to determine if it will execute properly.

In the first free-response question on the AP Exam—Methods and Control Structures—students will be instructed to write two methods or a constructor and one method of a given class based on provided specifications and examples. The method will require students to write iterative or conditional statements or both, as well as statements that call methods in a class. As described in Unit 1, one of the methods will involve calling `String` methods. Practice close reading techniques with students, such as underlining keywords, instance variables, return types, and parameters. These close reading techniques can help students process the questions quickly without inadvertently missing key information.

UNIT AT A GLANCE

Topic	Instructional Periods	Suggested Skills
2.1 Algorithms with Selection and Repetition	1	1.A Determine an appropriate program design to solve a problem or accomplish a task.
2.2 Boolean Expressions	1	3.A Determine the result or output based on statement execution order in an algorithm.
2.3 `if` Statements	3	2.A Write program code to implement an algorithm. 3.A Determine the result or output based on statement execution order in an algorithm. 4.A Describe the behavior of a code segment or program.
2.4 Nested `if` Statements	2	2.A Write program code to implement an algorithm. 4.A Describe the behavior of a code segment or program.
2.5 Compound Boolean Expressions	2	2.A Write program code to implement an algorithm. 3.A Determine the result or output based on statement execution order in an algorithm.
2.6 Comparing Boolean Expressions	2	2.A Write program code to implement an algorithm. 3.A Determine the result or output based on statement execution order in an algorithm.
2.7 `while` Loops	3	2.A Write program code to implement an algorithm. 3.A Determine the result or output based on statement execution order in an algorithm. 4.B Describe the initial conditions that must be met for a code segment to work as intended or described.
2.8 `for` Loops	3	2.A Write program code to implement an algorithm. 3.A Determine the result or output based on statement execution order in an algorithm. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.

continued on next page

UNIT AT A GLANCE *(cont'd)*

Topic	Instructional Periods	Suggested Skills
2.9 Implementing Selection and Iteration Algorithms	3	2.A Write program code to implement an algorithm. 3.A Determine the result or output based on statement execution order in an algorithm. 4.A Describe the behavior of a code segment or program.
2.10 Implementing String Algorithms	3	2.C Write program code involving procedural abstractions. 3.C Determine the result or output based on code that contains procedural abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.
2.11 Nested Iteration	2	2.A Write program code to implement an algorithm. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error. 4.B Describe the initial conditions that must be met for a code segment to work as intended or described.
2.12 Informal Run-Time Analysis	1	4.A Describe the behavior of a code segment or program.

Go to **AP Classroom** to assign the **Progress Checks** for Unit 2.
Review the results in class to identify and address any student misunderstandings.

SAMPLE INSTRUCTIONAL ACTIVITIES

The sample activities on this page are optional and are offered to provide possible ways to incorporate various instructional approaches into the classroom. Teachers do not need to use these activities or instructional approaches and are free to alter or edit them. The examples below were developed in partnership with teachers from the AP community to share ways that they approach teaching some of the topics in this unit. Please refer to the **Instructional Approaches** section beginning on p. 125 for more examples of activities and strategies.

Activity	Topic	Sample Activity
1	2.3	**Code Tracing** Provide students with several code segments that contain conditional statements. Have students trace various sample inputs, keeping track of the statements that get executed and the order in which they are executed. This can help students find errors and validate results.
2	2.2–2.6	**Pair Programming** Have students work with a partner to create a "guess checker" that could be used as part of a larger game. Students compare four given digits to a preexisting four-digit code that is stored in individual variables. Their program should provide output containing the number of correct digits in correct locations and the number of correct digits in incorrect locations. This program can be continually improved as students learn about nested conditional statements and compound Boolean expressions.
3	2.5	**Student Response System** Provide students with a code segment that utilizes conditional statements and a compound Boolean expression. Ask them to choose an equivalent code segment that uses a nested conditional statement (and vice versa). Have them report their responses using a student response system.
4	2.6	**Diagramming** Have students create truth tables by listing all the possible true and false combinations and corresponding Boolean values for a given compound Boolean expression. Then give them input values. Have students determine the value of each individual Boolean expression and then use the truth table to determine the result of the compound Boolean expression.
5	2.6	**Predict and Confirm** Have students predict the output of several different code segments that compare object references—some that use `.equals()` and some that use `==`. Once done, have them create a program that contains those code segments and compare the actual and expected results. This is best illustrated with a self-written simple class.
6	2.7–2.8	**Jigsaw** As a whole class, look at a code segment containing iteration and the resulting output. Then divide students into groups, and provide each group with a slightly modified code segment. After groups have determined how the result changes based on their modified segment, have them get together with students who investigated a different version of the code segment and share their conclusions.

Selection and Iteration

Activity	Topic	Sample Activity
7	2.9	**Marking the Text** Provide students with a code segment that, when given an integer `n`, determines the nth word from a string of words separated by spaces. Have them annotate what each statement does in the code segment. Then ask students to use their annotated code segment as a guide to write a similar code segment that, given a student number as input, returns the name of a student from a `String` containing the first name of all students in the class, each separated by a space.
8	2.12	**Simplify the Problem** Provide students with two code segments with similar iterative structures but with different loop bounds. Have them trace through the execution of each loop to see the differences.

{ } After completing this unit, students will have covered all of the necessary content for the Magpie 2.0 Lab, which can be found in **AP Classroom**.

TOPIC 2.1
Algorithms with Selection and Repetition

Required Course Content

LEARNING OBJECTIVE

2.1.A

Represent patterns and algorithms that involve selection and repetition found in everyday life using written language or diagrams.

ESSENTIAL KNOWLEDGE

2.1.A.1

The building blocks of algorithms include sequencing, selection, and repetition.

2.1.A.2

Algorithms can contain selection, through decision making, and repetition, via looping.

2.1.A.3

Selection occurs when a choice of how the execution of an algorithm will proceed is based on a true or false decision.

2.1.A.4

Repetition is when a process repeats itself until a desired outcome is reached.

2.1.A.5

The order in which sequencing, selection, and repetition are used contributes to the outcome of the algorithm.

SUGGESTED SKILLS

1.A

Determine an appropriate program design to solve a problem or accomplish a task.

SUGGESTED SKILLS

3.A

Determine the result or output based on statement execution order in an algorithm.

TOPIC 2.2
Boolean Expressions

Required Course Content

LEARNING OBJECTIVE

2.2.A

Develop code to create Boolean expressions with relational operators and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

2.2.A.1

Values can be compared using the *relational operators* `==` and `!=` to determine whether the values are the same. With primitive types, this compares the actual primitive values. With reference types, this compares the object references.

2.2.A.2

Numeric values can be compared using the relational operators `<`, `>`, `<=`, and `>=` to determine the relationship between the values.

2.2.A.3

An expression involving relational operators evaluates to a Boolean value.

TOPIC 2.3
`if` Statements

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

3.A
Determine the result or output based on statement execution order in an algorithm.

4.A
Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE

2.3.A
Develop code to represent branching logical processes by using selection statements and determine the result of these processes.

ESSENTIAL KNOWLEDGE

2.3.A.1
Selection statements change the sequential execution of statements.

2.3.A.2
An `if` statement is a type of selection statement that affects the flow of control by executing different segments of code based on the value of a Boolean expression.

2.3.A.3
A *one-way selection* (`if` statement) is used when there is a segment of code to execute under a certain condition. In this case, the body is executed only when the Boolean expression is `true`.

2.3.A.4
A *two-way selection* (`if-else` statement) is used when there are two segments of code—one to be executed when the Boolean expression is `true` and another segment for when the Boolean expression is `false`. In this case, the body of the `if` is executed when the Boolean expression is `true`, and the body of the `else` is executed when the Boolean expression is `false`.

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

4.A
Describe the behavior of a code segment or program.

TOPIC 2.4
Nested `if` Statements

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
2.4.A Develop code to represent nested branching logical processes and determine the result of these processes.	**2.4.A.1** Nested `if` statements consist of `if`, `if-else`, or `if-else-if` statements within `if`, `if-else`, or `if-else-if` statements. **2.4.A.2** The Boolean expression of the inner nested `if` statement is evaluated only if the Boolean expression of the outer `if` statement evaluates to `true`. **2.4.A.3** A *multiway selection* (`if-else-if`) is used when there are a series of expressions with different segments of code for each condition. Multiway selection is performed such that no more than one segment of code is executed based on the first expression that evaluates to `true`. If no expression evaluates to `true` and there is a trailing `else` statement, then the body of the `else` is executed.

TOPIC 2.5
Compound Boolean Expressions

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

3.A
Determine the result or output based on statement execution order in an algorithm.

Required Course Content

LEARNING OBJECTIVE

2.5.A
Develop code to represent compound Boolean expressions and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

2.5.A.1
Logical operators `!` (not), `&&` (and), and `||` (or) are used with Boolean expressions. The expression `!a` evaluates to `true` if `a` is `false` and evaluates to `false` otherwise. The expression `a && b` evaluates to `true` if both `a` and `b` are `true` and evaluates to `false` otherwise. The expression `a || b` evaluates to `true` if `a` is `true`, `b` is `true`, or both, and evaluates to `false` otherwise. The order of precedence for evaluating logical operators is `!` (not), `&&` (and), then `||` (or). An expression involving logical operators evaluates to a Boolean value.

2.5.A.2
Short-circuit evaluation occurs when the result of a logical operation using `&&` or `||` can be determined by evaluating only the first Boolean expression. In this case, the second Boolean expression is not evaluated.

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

3.A
Determine the result or output based on statement execution order in an algorithm.

TOPIC 2.6
Comparing Boolean Expressions

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
2.6.A Compare equivalent Boolean expressions.	**2.6.A.1** Two Boolean expressions are *equivalent* if they evaluate to the same value in all cases. Truth tables can be used to prove Boolean expressions are equivalent.
	2.6.A.2 *De Morgan's law* can be applied to Boolean expressions to create equivalent Boolean expressions. Under De Morgan's law, the Boolean expression `!(a && b)` is equivalent to `!a || !b` and the Boolean expression `!(a || b)` is equivalent to `!a && !b`.
2.6.B Develop code to compare object references using Boolean expressions and determine the result of these expressions.	**2.6.B.1** Two different variables can hold references to the same object. Object references can be compared using `==` and `!=`.
	2.6.B.2 An object reference can be compared with `null`, using `==` or `!=`, to determine if the reference actually references an object.
	2.6.B.3 Classes often define their own `equals` method, which can be used to specify the criteria for equivalency for two objects of the class. The equivalency of two objects is most often determined using attributes from the two objects.

EXCLUSION STATEMENT—*Overriding the* `equals` *method is outside the scope of the AP Computer Science A course and exam.*

TOPIC 2.7
`while` Loops

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

3.A
Determine the result or output based on statement execution order in an algorithm.

4.B
Describe the initial conditions that must be met for a code segment to work as intended or described.

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
2.7.A Identify when an iterative process is required to achieve a desired result.	**2.7.A.1** *Iteration* is a form of repetition. Iteration statements change the flow of control by repeating a segment of code zero or more times as long as the Boolean expression controlling the loop evaluates to `true`.
	2.7.A.2 An *infinite loop* occurs when the Boolean expression in an iterative statement always evaluates to `true`.
	2.7.A.3 The loop body of an iterative statement will not execute if the Boolean expression initially evaluates to `false`.
	2.7.A.4 *Off by one* errors occur when the iteration statement loops one time too many or one time too few.
2.7.B Develop code to represent iterative processes using `while` loops and determine the result of these processes.	**2.7.B.1** A `while` loop is a type of iterative statement. In `while` loops, the Boolean expression is evaluated before each iteration of the loop body, including the first. When the expression evaluates to `true`, the loop body is executed. This continues until the Boolean expression evaluates to `false`, whereupon the iteration terminates.

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

3.A
Determine the result or output based on statement execution order in an algorithm.

3.D
Explain why a code segment will not compile or work as intended and modify the code to correct the error.

TOPIC 2.8
`for` Loops

Required Course Content

LEARNING OBJECTIVE

2.8.A
Develop code to represent iterative processes using `for` loops and determine the result of these processes.

ESSENTIAL KNOWLEDGE

2.8.A.1
A `for` loop is a type of iterative statement. There are three parts in a `for` loop header: the initialization, the Boolean expression, and the update.

2.8.A.2
In a `for` loop, the initialization statement is only executed once before the first Boolean expression evaluation. The variable being initialized is referred to as a *loop control variable*. The Boolean expression is evaluated immediately after the loop control variable is initialized and then following each execution of the increment statement until it is `false`. In each iteration, the update is executed after the entire loop body is executed and before the Boolean expression is evaluated again.

2.8.A.3
A `for` loop can be rewritten into an equivalent `while` loop (and vice versa).

TOPIC 2.9
Implementing Selection and Iteration Algorithms

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

3.A
Determine the result or output based on statement execution order in an algorithm.

4.A
Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE

2.9.A
Develop code for standard and original algorithms (without data structures) and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE

2.9.A.1
There are standard algorithms to:

- identify if an integer is or is not evenly divisible by another integer
- identify the individual digits in an integer
- determine the frequency with which a specific criterion is met
- determine a minimum or maximum value
- compute a sum or average

SUGGESTED SKILLS

2.C
Write program code involving procedural abstractions.

3.C
Determine the result or output based on code that contains procedural abstractions.

3.D
Explain why a code segment will not compile or work as intended and modify the code to correct the error.

TOPIC 2.10
Implementing String Algorithms

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
2.10.A Develop code for standard and original algorithms that involve strings and determine the result of these algorithms.	**2.10.A.1** There are standard string algorithms to: • find if one or more substrings have a particular property • determine the number of substrings that meet specific criteria • create a new string with the characters reversed

TOPIC 2.11
Nested Iteration

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
2.11.A Develop code to represent nested iterative processes and determine the result of these processes.	2.11.A.1 *Nested iteration statements* are iteration statements that appear in the body of another iteration statement. When a loop is nested inside another loop, the inner loop must complete all its iterations before the outer loop can continue to its next iteration.

SUGGESTED SKILLS

2.A
Write program code to implement an algorithm.

3.D
Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.B
Describe the initial conditions that must be met for a code segment to work as intended or described.

SUGGESTED SKILLS

4.A

Describe the behavior of a code segment or program.

TOPIC 2.12
Informal Run-Time Analysis

Required Course Content

LEARNING OBJECTIVE

2.12.A

Calculate statement execution counts and informal run-time comparison of iterative statements.

ESSENTIAL KNOWLEDGE

2.12.A.1

A *statement execution count* indicates the number of times a statement is executed by the program. Statement execution counts are often calculated informally through tracing and analysis of the iterative statements.

AP COMPUTER SCIENCE A

UNIT 3

Class Creation

10–18%
AP EXAM WEIGHTING

~20–22
CLASS PERIODS

Remember to go to **AP Classroom** to assign students the online **Progress Checks** for this unit.

Whether assigned as homework or completed in class, the **Progress Checks** provide each student with immediate feedback related to this unit's topics and skills.

Progress Check Unit 3 Part 1: Topics 3.1–3.4

Multiple-choice: ~12 questions

Progress Check Unit 3 Part 2: Topics 3.5–3.9

Multiple-choice: ~15 questions
Free-response: 2 questions

- Class Design

UNIT 3

10–18% AP EXAM WEIGHTING | **~20–22** CLASS PERIODS

Class Creation

ESSENTIAL QUESTIONS

- How can using a model of traffic patterns improve travel time?
- How can programs be written to protect your bank account balance from being inadvertently changed?
- How can you be a responsible programmer?

Developing Understanding

This unit will pull together information from the previous two units to create new, user-defined reference data types in the form of classes. The ability to accurately model real-world entities in a computer program is a large part of what makes computer science so powerful. By being able to design their own classes, programmers are not limited to the existing classes provided within the Java libraries and can therefore represent their own ideas through classes. This unit focuses on identifying appropriate behaviors and attributes of real-world entities and organizing these into classes and their corresponding methods. The creation of computer programs can have an extensive impact on societies, economies, and cultures. The legal and ethical concerns that come with programs and the responsibilities of programmers are also addressed in this unit.

Building Computational Thinking Practices

2.B 2.C 3.B

Programmers often rely on existing program code to build new programs. Using existing code saves time because it has already been tested. By using built-in Java classes such as `String` and `Math` or user-defined classes, students learn how to interact with and utilize those classes to create objects and call methods. Understanding how to write program code that uses those processes allows students to write programs that solve a wider variety of problems. Deciding how to design a class so it adheres to the desired specifications is an important skill for programmers. **(2.B, 2.C)**

Spending time analyzing existing program code allows students to better understand how methods are called and how classes are designed. Students should practice determining the number of times a given loop structure will execute as well as the implications associated with code changes, such as how using a different iterative structure might change the result of a set of program code. **(3.B)**

Preparing for the AP Exam

The ability to create and use instance and class methods is assessed in both the multiple-choice and free-response sections of the exam. Students should know how to look at a class definition and determine if the class is syntactically correct in terms of its instance variables, constructors, and methods. Once the new class is designed, students should be able to implement program code to create and use that new class. The behaviors of an object are expressed through writing methods in the class, which include expressions, conditional statements, and iterative statements.

In the second free-response question on the AP Exam—Class Design—students will be provided with a scenario and specifications in the form of a table demonstrating ways to interact with the class and those results. A second class might also be included. Students will be instructed to design and implement a class based on provided specifications and examples. The class must include class header, private instance variables, constructors, methods, and implementation

of the constructor and required methods. Therefore, students must have many opportunities throughout the course to design their own classes based on program specifications and observations of real-world objects. This process includes identifying the attributes (data) that define an object of a class and the behaviors (methods) that define what an object of a class does.

UNIT AT A GLANCE

Topic	Instructional Periods	Suggested Skills
3.1 Abstraction and Program Design	2	1.A Determine an appropriate program design to solve a problem or accomplish a task.
3.2 Impact of Program Design	1	5.A Explain how computing impacts society, economy, and culture.
3.3 Anatomy of a Class	2	1.A Determine an appropriate program design to solve a problem or accomplish a task. 2.B Write program code involving data abstractions.
3.4 Constructors	2	2.B Write program code involving data abstractions. 3.B Determine the result or output based on code that contains data abstractions.
3.5 Methods: How to Write Them	4	2.C Write program code involving procedural abstractions. 3.C Determine the result or output based on code that contains procedural abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error. 4.B Describe the initial conditions that must be met for a code segment to work as intended or described.
3.6 Methods: Passing and Returning References of an Object	2	2.C Write program code involving procedural abstractions. 4.A Describe the behavior of a code segment or program.
3.7 Class Variables and Methods	2	2.C Write program code involving procedural abstractions. 3.B Determine the result or output based on code that contains data abstractions.

continued on next page

UNIT AT A GLANCE *(cont'd)*

Topic	Instructional Periods	Suggested Skills
3.8 Scope and Access	2	3.B Determine the result or output based on code that contains data abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error. 4.A Describe the behavior of a code segment or program.
3.9 `this` Keyword	1	2.B Write program code involving data abstractions.

Go to **AP Classroom** to assign the **Progress Checks** for Unit 3.
Review the results in class to identify and address any student misunderstandings.

SAMPLE INSTRUCTIONAL ACTIVITIES

The sample activities on this page are optional and are offered to provide possible ways to incorporate various instructional approaches into the classroom. Teachers do not need to use these activities or instructional approaches and are free to alter or edit them. The examples below were developed in partnership with teachers from the AP community to share ways that they approach teaching some of the topics in this unit. Please refer to the **Instructional Approaches** section beginning on p. 125 for more examples of activities and strategies.

Activity	Topic	Sample Activity
1	3.1	**Kinesthetic Learning** Have students break into groups of 4–5 to play board games for about 10 minutes. While they play the game, they should keep track of the various nouns they encounter and actions that happen as part of the game. The nouns can be represented in the computer as classes, and the actions are the behaviors. At the end of game play, ask students to create UML (Unified Modeling Language) diagrams for the identified classes.
2	3.5	**Marking the Text** Present students with specifications, and have them highlight or underline any preconditions (both implicit and explicit) that exist for the method to function. This includes information about parameters, such as object references not being `null`.
3	3.5–3.6	**Create a Plan** When asked to write a method, have students write an outline using pseudocode with paper and pencil. Then, go through it step-by-step with sample input to ensure the process is correct and to determine if any additional information is needed before beginning to program a solution on the computer.
4	3.7	**Paraphrasing** Provide students with several example classes that utilize class variables for unique identification numbers or for counting the number of objects that have been created, but do not provide any description or documentation for the code. Have students spend time creating objects and calling the class methods to investigate how the class variables behave. Then have them document the code appropriately to describe how each class utilizes class variables and methods.

{ } After completing this unit, students will have covered all of the necessary content for the Virtual Pet Lab, which can be found in **AP Classroom**.

SUGGESTED SKILLS

1.A

Determine an appropriate program design to solve a problem or accomplish a task.

TOPIC 3.1
Abstraction and Program Design

Required Course Content

LEARNING OBJECTIVE

3.1.A

Represent the design of a program by using natural language or creating diagrams that indicate the classes in the program and the data and procedural abstractions found in each class by including all attributes and behaviors.

ESSENTIAL KNOWLEDGE

3.1.A.1

Abstraction is the process of reducing complexity by focusing on the main idea. By hiding details irrelevant to the question at hand and bringing together related and useful details, abstraction reduces complexity and allows one to focus on the idea.

3.1.A.2

Data abstraction provides a separation between the abstract properties of a data type and the concrete details of its representation. Data abstraction manages complexity by giving data a name without referencing the specific details of the representation. Data can take the form of a single variable or a collection of data, such as in a class or a set of data.

3.1.A.3

An *attribute* is a type of data abstraction that is defined in a class outside any method or constructor. An *instance variable* is an attribute whose value is unique to each instance of the class. A *class variable* is an attribute shared by all instances of the class.

3.1.A.4

Procedural abstraction provides a name for a process and allows a method to be used only knowing what it does, not how it does it. Through *method decomposition*, a programmer breaks down larger behaviors of the class into smaller behaviors by creating methods to represent each individual smaller behavior. A procedural abstraction may extract shared features to generalize functionality instead of duplicating code. This allows for code reuse, which helps manage complexity.

continued on next page

LEARNING OBJECTIVE

3.1.A

Represent the design of a program by creating diagrams that indicate the classes in the program and the data and procedural abstractions found in each class by including all attributes and behaviors.

ESSENTIAL KNOWLEDGE

3.1.A.5

Using parameters allows procedures to be generalized, enabling the procedures to be reused with a range of input values or arguments.

3.1.A.6

Using procedural abstraction in a program allows programmers to change the internals of a method (to make it faster, more efficient, use less storage, etc.) without needing to notify method users of the change as long as the method signature and what the method does is preserved.

3.1.A.7

Prior to implementing a class, it is helpful to take time to design each class including its attributes and behaviors. This design can be represented using natural language or diagrams.

SUGGESTED SKILLS

5.A

Explain how computing impacts society, economy, and culture.

TOPIC 3.2
Impact of Program Design

Required Course Content

LEARNING OBJECTIVE

3.2.A

Explain the social and ethical implications of computing systems.

ESSENTIAL KNOWLEDGE

3.2.A.1

System reliability refers to the program being able to perform its tasks as expected under stated conditions without failure. Programmers should make an effort to maximize system reliability by testing the program with a variety of conditions.

3.2.A.2

The creation of programs has impacts on society, the economy, and culture. These impacts can be both beneficial and harmful. Programs meant to fill a need or solve a problem can have unintended harmful effects beyond their intended use.

3.2.A.3

Legal issues and intellectual property concerns arise when creating programs. Programmers often reuse code written by others and published as open source and free to use. Incorporation of code that is not published as open source requires the programmer to obtain permission and often purchase the code before integrating it into their program.

TOPIC 3.3
Anatomy of a Class

SUGGESTED SKILLS

1.A
Determine an appropriate program design to solve a problem or accomplish a task.

2.B
Write program code involving data abstractions.

Required Course Content

LEARNING OBJECTIVE

3.3.A
Develop code to designate access and visibility constraints to classes, data, constructors, and methods.

ESSENTIAL KNOWLEDGE

3.3.A.1
Data encapsulation is a technique in which the implementation details of a class are kept hidden from external classes. The keywords `public` and `private` affect the access of classes, data, constructors, and methods. The keyword `private` restricts access to the declaring class, while the keyword `public` allows access from classes outside the declaring class.

3.3.A.2
In this course, classes are always designated `public` and are declared with the keyword `class`.

3.3.A.3
In this course, constructors are always designated `public`.

3.3.A.4
Instance variables belong to the object, and each object has its own copy of the variable.

3.3.A.5
Access to attributes should be kept internal to the class in order to accomplish encapsulation. Therefore, it is good programming practice to designate the instance variables for these attributes as `private` unless the class specification states otherwise.

3.3.A.6
Access to behaviors can be internal or external to the class. Methods designated as `public` can be accessed internally or externally to a class, whereas methods designated as `private` can only be accessed internally to the class.

SUGGESTED SKILLS

2.B
Write program code involving data abstractions.

3.B
Determine the result or output based on code that contains data abstractions.

TOPIC 3.4
Constructors

Required Course Content

LEARNING OBJECTIVE

3.4.A
Develop code to declare instance variables for the attributes to be initialized in the body of the constructors of a class.

ESSENTIAL KNOWLEDGE

3.4.A.1
An object's *state* refers to its attributes and their values at a given time and is defined by instance variables belonging to the object. This defines a *has-a* relationship between the object and its instance variables.

3.4.A.2
A constructor is used to set the initial state of an object, which should include initial values for all instance variables. When a constructor is called, memory is allocated for the object and the associated object reference is returned. Constructor parameters, if specified, provide data to initialize instance variables.

3.4.A.3
When a mutable object is a constructor parameter, the instance variable should be initialized with a copy of the referenced object. In this way, the instance variable does not hold a reference to the original object, and methods are prevented from modifying the state of the original object.

3.4.A.4
When no constructor is written, Java provides a no-parameter constructor, and the instance variables are set to default values according to the data type of the attribute. This constructor is called the *default constructor*.

3.4.A.5
The default value for an attribute of type `int` is `0`. The default value of an attribute of type `double` is `0.0`. The default value of an attribute of type `boolean` is `false`. The default value of a reference type is `null`.

TOPIC 3.5
Methods: How to Write Them

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
3.5.A Develop code to define behaviors of an object through methods written in a class using primitive values and determine the result of calling these methods.	**3.5.A.1** A `void` method does not return a value. Its header contains the keyword `void` before the method name. **3.5.A.2** A non-void method returns a single value. Its header includes the return type in place of the keyword `void`. **3.5.A.3** In non-void methods, a return expression compatible with the return type is evaluated, and the value is returned. This is referred to as *return by value*. **3.5.A.4** The `return` keyword is used to return the flow of control to the point where the method or constructor was called. Any code that is sequentially after a return statement will never be executed. Executing a return statement inside a selection or iteration statement will halt the statement and exit the method or constructor. **3.5.A.5** An *accessor method* allows objects of other classes to obtain a copy of the value of instance variables or class variables. An accessor method is a non-void method. **3.5.A.6** A *mutator (modifier) method* is a method that changes the values of the instance variables or class variables. A mutator method is often a void method.

continued on next page

SUGGESTED SKILLS

2.C Write program code involving procedural abstractions.

3.C Determine the result or output based on code that contains procedural abstractions.

3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.B Describe the initial conditions that must be met for a code segment to work as intended or described.

Class Creation

LEARNING OBJECTIVE

3.5.A

Develop code to define behaviors of an object through methods written in a class using primitive values and determine the result of calling these methods.

ESSENTIAL KNOWLEDGE

3.5.A.7

Methods with parameters receive values through those parameters and use those values in accomplishing the method's task.

3.5.A.8

When an argument is a primitive value, the parameter is initialized with a copy of that value. Changes to the parameter have no effect on the corresponding argument.

TOPIC 3.6
Methods: Passing and Returning References of an Object

Required Course Content

LEARNING OBJECTIVE

3.6.A

Develop code to define behaviors of an object through methods written in a class using object references and determine the result of calling these methods.

ESSENTIAL KNOWLEDGE

3.6.A.1

When an argument is an object reference, the parameter is initialized with a copy of that reference; it does not create a new independent copy of the object. If the parameter refers to a mutable object, the method or constructor can use this reference to alter the state of the object. It is good programming practice to not modify mutable objects that are passed as parameters unless required in the specification.

3.6.A.2

When the return expression evaluates to an object reference, the reference is returned, not a reference to a new copy of the object.

3.6.A.3

Methods cannot access the private data and methods of a parameter that holds a reference to an object unless the parameter is the same type as the method's enclosing class.

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

4.A

Describe the behavior of a code segment or program.

SUGGESTED SKILLS

2.C
Write program code involving procedural abstractions.

3.B
Determine the result or output based on code that contains data abstractions.

TOPIC 3.7
Class Variables and Methods

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
3.7.A Develop code to define behaviors of a class through class methods.	**3.7.A.1** Class methods cannot access or change the values of instance variables or call instance methods without being passed an instance of the class via a parameter.
	3.7.A.2 Class methods can access or change the values of class variables and can call other class methods.
3.7.B Develop code to declare the class variables that belong to the class.	**3.7.B.1** Class variables belong to the class, with all objects of a class sharing a single copy of the class variable. Class variables are designated with the `static` keyword before the variable type.
	3.7.B.2 Class variables that are designated `public` are accessed outside of the class by using the class name and the dot operator, since they are associated with a class, not objects of a class.
	3.7.B.3 When a variable is declared `final`, its value cannot be modified.

TOPIC 3.8
Scope and Access

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
3.8.A Explain where variables can be used in the code.	**3.8.A.1** *Local variables* are variables declared in the headers or bodies of blocks of code. Local variables can only be accessed in the block in which they are declared. Since constructors and methods are blocks of code, parameters to constructors or methods are also considered local variables. These variables may only be used within the constructor or method and cannot be declared to be `public` or `private`.
	3.8.A.2 When there is a local variable or parameter with the same name as an instance variable, the variable name will refer to the local variable instead of the instance variable within the body of the constructor or method.

SUGGESTED SKILLS

3.B Determine the result or output based on code that contains data abstractions.

3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A Describe the behavior of a code segment or program.

SUGGESTED SKILLS

2.B

Write program code involving procedural abstractions.

TOPIC 3.9
`this` Keyword

Required Course Content

LEARNING OBJECTIVE

3.9.A

Develop code for expressions that are self-referencing and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

3.9.A.1

Within an instance method or a constructor, the keyword `this` acts as a special variable that holds a reference to the current object—the object whose method or constructor is being called.

3.9.A.2

The keyword `this` can be used to pass the current object as an argument in a method call.

3.9.A.3

Class methods do not have a `this` reference.

AP COMPUTER SCIENCE A

UNIT 4

Data Collections

30–40%
AP EXAM WEIGHTING

~50–52
CLASS PERIODS

Remember to go to **AP Classroom** to assign students the online **Progress Checks** for this unit.

Whether assigned as homework or completed in class, the **Progress Checks** provide each student with immediate feedback related to this unit's topics and skills.

Progress Check Unit 4 Part 1: Topics 4.1–4.5

Multiple-choice: ~18 questions
Free-response: 2 questions

- Data Analysis with Array (partial)

Progress Check Unit 4 Part 2: Topics 4.6–4.10

Multiple-choice: ~21 questions
Free-response: 2 questions

- Data Analysis with `ArrayList`

Progress Check Unit 4 Part 3: Topics 4.11–4.17

Multiple-choice: ~21 questions
Free-response: 2 questions

- 2D Array

UNIT 4

30–40% AP EXAM WEIGHTING | **~50–52** CLASS PERIODS

Data Collections

ESSENTIAL QUESTIONS

- What type of personal data is being collected when using a food delivery app, and how does the app collect the data?
- How can programs leverage volcano data to make predictions about the next eruption?
- Why is an `ArrayList` more appropriate for storing your music playlist, while an array is more appropriate for storing your class schedule?
- What are some real-world processes that are recursive in nature?

Developing Understanding

This unit introduces the data structures array, `ArrayList`, and 2D array, which are used to represent collections of related data using a single variable rather than multiple variables. Arrays have a static size, which causes limitations related to the number of elements stored, and it can be challenging to reorder elements stored in arrays. An `ArrayList` object has a dynamic size, and the class contains methods for insertion and deletion of elements, making reordering and shifting items easier. Deciding which data structure to select becomes increasingly important as the size of the data set grows, such as when using a large real-world data set. A 2D array is most suitable to represent a table. Unlike 1D arrays, 2D arrays require nested iterative statements to traverse and access all elements. The easiest way to accomplish this is in row-major order, but it is important to discuss additional traversal patterns, such as column-major or back and forth. Just as there are useful standard algorithms when dealing with primitive data, there are standard algorithms to use with data structures. Additional algorithms, such as two common searching and three common sorting algorithms, are also covered.

Sometimes a problem can be solved by solving smaller or simpler versions of the same problem rather than attempting an iterative solution. This is called recursion, and it is a powerful math and computer science concept. In this unit, students will learn how to write simple recursive methods and determine the purpose or output of a recursive method by tracing. Students will revisit how control is passed when methods are called, which is necessary knowledge when working with recursion. Tracing skills are also helpful.

Also in this unit, students will learn about privacy concerns related to storing large amounts of personal data and about what can happen if such information is compromised. They will also examine the potential for bias in collecting and using data.

Building Computational Thinking Practices

1.B 4.B 5.A

Having students practice writing programs for data sets of undetermined sizes provides a relevant and realistic experience with data. This requires students to not only determine what data collection to use but also what information can be gained from that data set. Students should focus more on the algorithm and ensuring that it will work in all situations rather than on an individual result. With larger data sets, programmers must also consider the amount of time it will take for their program code to run. **(1.B)**

Students must understand that methods only work as designed when their preconditions are met. If those preconditions are not checked, then the method is not guaranteed to work as intended. Help students learn this by showing them the results of invalid calls to methods when their preconditions have not been met. **(4.B)**

Data Collections

While programs are typically designed to achieve a specific purpose, they may have unintended consequences that can impact society as a whole. Students should be prepared to explain the risks to privacy from collecting and storing personal data on computer systems and how these impacts can be beneficial and/or harmful. **(5.A)**

Preparing for the AP Exam

On both the multiple-choice and free-response sections of the AP Exam, students need to know how to traverse arrays, `ArrayLists`, and 2D arrays in a variety of ways (e.g., from the first element to the last element, from the last element to the first element, and every other element). Knowing various iterative structures can be helpful when students use tracing to determine what program code is doing. Students can follow this traversal for the first few iterations and apply that pattern to the remaining elements in the array. When an array is not specifically defined in a code segment, students can create their own small-sized array and use it to trace the code that is given. Recursion is assessed only through the multiple-choice section of the exam. Students are often asked to determine the result of a specific call to a recursive method or to describe the behavior of a recursive method. A call to a recursive method is just like a call to a non-recursive method. Because a method is an abstraction of a process that has a specific result for each varied input, using a specific input value provides insight into how the method functions for that input. By understanding several instances of the method call, students can abstract and generalize the method's overall purpose or process.

In the third free-response question on the AP Exam—Data Analysis with `ArrayList`—students are provided with an outline of a data set that has been represented by a class, and the question requires analysis of an `ArrayList` of objects of this class. Students will be instructed to write one method of a given class based on provided specifications and examples. This method requires students to use, analyze, and manipulate data in an `ArrayList` structure. When writing solutions, students are often asked to insert or remove elements from the `ArrayList`. In these cases, adjustments will need to be made to the loop counter to account for skipping an element or attempting to access elements that no longer exist.

In the fourth free-response question—2D Array—students are provided with a scenario and its associated classes. Students will be instructed to write one method of a given class based on provided specifications and examples. The method requires students to access or manipulate data in a 2D array structure. While there is a specific nested structure to traverse elements in a 2D array in row-major order, this structure can be modified to traverse 2D arrays in other ways, such as column-major, by switching the nested iterative statements. Additional modifications can be made to traverse rows or columns in different ways, such as back and forth or up and down. However, when making these adjustments, students need to remember to adjust the bounds of the iterative statements appropriately. To prepare for this type of free-response question, students should practice traversing 2D arrays in these nonstandard ways, being sure to test the boundary conditions of the iterative statements.

UNIT AT A GLANCE

Topic	Instructional Periods	Suggested Skills
4.1 Ethical and Social Issues Around Data Collection	1	1.B Determine what knowledge can be extracted from data. 5.A Explain how computing impacts society, economy, and culture.
4.2 Introduction to Using Data Sets	1	1.B Determine what knowledge can be extracted from data.
4.3 Array Creation and Access	2	2.B Write program code involving data abstractions. 3.B Determine the result or output based on code that contains data abstractions.
4.4 Array Traversals	3	2.B Write program code involving data abstractions. 3.B Determine the result or output based on code that contains data abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.
4.5 Implementing Array Algorithms	4	2.B Write program code involving data abstractions. 3.B Determine the result or output based on code that contains data abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error. 4.A Describe the behavior of a code segment or program.
4.6 Using Text Files	3	2.C Write program code involving procedural abstractions. 3.C Determine the result or output based on code that contains procedural abstractions. 4.B Describe the initial conditions that must be met for a code segment to work as intended or described.
4.7 Wrapper Classes	1	2.B Write program code involving data abstractions.

continued on next page

UNIT AT A GLANCE *(cont'd)*

Topic	Instructional Periods	Suggested Skills
4.8 `ArrayList` Methods	3	2.B Write program code involving data abstractions. 3.B Determine the result or output based on code that contains data abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.
4.9 `ArrayList` Traversals	3	2.B Write program code involving data abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error. 4.A Describe the behavior of a code segment or program.
4.10 Implementing `ArrayList` Algorithms	4	2.B Write program code involving data abstractions. 3.B Determine the result or output based on code that contains data abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error. 4.A Describe the behavior of a code segment or program.
4.11 2D Array Creation and Access	2	2.B Write program code involving data abstractions. 3.B Determine the result or output based on code that contains data abstractions.
4.12 2D Array Traversals	3	2.B Write program code involving data abstractions. 3.B Determine the result or output based on code that contains data abstractions. 3.D Explain why a code segment will not compile or work as intended and modify the code to correct the error.

continued on next page

UNIT AT A GLANCE *(cont'd)*

Topic	Instructional Periods	Suggested Skills
4.13 Implementing 2D Array Algorithms	4	**2.B** Write program code involving data abstractions. **3.B** Determine the result or output based on code that contains data abstractions. **3.D** Explain why a code segment will not compile or work as intended and modify the code to correct the error. **4.A** Describe the behavior of a code segment or program.
4.14 Searching Algorithms	3	**2.B** Write program code involving data abstractions. **3.B** Determine the result or output based on code that contains data abstractions. **4.A** Describe the behavior of a code segment or program.
4.15 Sorting Algorithms	3	**3.B** Determine the result or output based on code that contains data abstractions. **4.A** Describe the behavior of a code segment or program. **4.B** Describe the initial conditions that must be met for a code segment to work as intended or described.
4.16 Recursion	3	**3.C** Determine the result or output based on code that contains procedural abstractions. **4.A** Describe the behavior of a code segment or program. **4.B** Describe the initial conditions that must be met for a code segment to work as intended or described.
4.17 Recursive Searching and Sorting	2	**4.A** Describe the behavior of a code segment or program. **4.B** Describe the initial conditions that must be met for a code segment to work as intended or described.

Go to **AP Classroom** to assign the **Progress Checks** for Unit 4.
Review the results in class to identify and address any student misunderstandings

SAMPLE INSTRUCTIONAL ACTIVITIES

The sample activities on this page are optional and are offered to provide possible ways to incorporate various instructional approaches into the classroom. Teachers do not need to use these activities or instructional approaches and are free to alter or edit them. The examples below were developed in partnership with teachers from the AP community to share ways that they approach teaching some of the topics in this unit. Please refer to the **Instructional Approaches** section beginning on p. 125 for more examples of activities and strategies.

Activity	Topic	Sample Activity
1	4.3	**Diagramming** Provide students with several prompts to create and access elements in an array. After they determine the code for each prompt, have students draw a memory diagram that shows references and the arrays they point to. Have students update the diagram with each statement to demonstrate how changing the contents through one array reference effects all the array references for this array.
2	4.4	**Error Analysis** Provide students with several error-ridden code segments containing array traversals along with the expected output of each segment. Ask them to identify any errors that they see on paper and to suggest fixes to provide the expected output. Have them type up their solutions in an IDE to verify their work.
3	4.5	**Think-Pair-Share** Ask students to consider two program code segments that are meant to yield the same result: one using an indexed `for` loop and one using an enhanced for loop. Give them a few minutes to think independently about whether the two segments accomplish the same result and, if not, what changes could be made for that to happen. Then have them work with partners to come up with situations where it would make sense to use one type of loop over the other before sharing with the whole class.
4	4.5	**Pair Programming** Have students use pair programming to solve an array-based free-response question from exam years prior to 2026. Have one student be the driver for Part A while the other navigates, and have them switch for Part B. Once they are done, have partners exchange their solutions with another group and work through the scoring guidelines to "grade" that solution. Spend time as a class discussing the different approaches students used.
5	4.10	**Predict and Confirm** Have students look at the code they wrote to solve the free-response question for arrays on paper, and have them rewrite it using an `ArrayList`. Have them highlight the parts that need to be changed and determine how to change them. Then, have students type up the changes in an IDE and confirm that the program still works as expected.
6	4.8–4.10	**Identify a Subtask** Have students read through an `ArrayList`-based free-response question in groups and identify all subtasks. These subtasks could be conditional statements, iteration, or even other methods. Divide the subtasks among the group members, and have students implement their given subtask. When all students are finished, have them combine the subtasks into a single solution.

Activity	Topic	Sample Activity
7	4.10	**Discussion Group** Discuss the algorithm necessary to search for the smallest value in an `ArrayList`. Without any explanation, change the Boolean expression so it will find the largest value, and ask students to describe what the resulting algorithm will do. Then, change the algorithm to store and return the location of the largest value, and discuss the change.
8	4.11	**Using Manipulatives** Use different-sized egg cartons or ice cube trays with random compartments filled with small toys or candy. Create laminated cards with the code for the construction of and access to a 2D array, leaving blanks for the name and size dimensions. Have students fill in the missing code that would be used to represent the physical 2D array objects and access the randomly stored elements.
9	4.12	**Activating Prior Knowledge** When introducing 2D arrays and row-major traversal, ask students which part of the nested `for` loop structure loops over a 1D array. Based on what they know about the traversal of 1D array structures, ask them to calculate the number of times the inner loop executes.
10	4.13–4.15	**Sharing and Responding** As a class, create a set of test cases to be used with answers to a free-response question from this unit. Have students write their answers to the free-response question individually on paper. After exchanging solutions with another student, ask students to find errors or validate results of their peers' code by tracing the code with the developed test cases. Allow students an opportunity to provide feedback on the program code as well as the results of each test case.
11	4.16	**Sharing and Responding** Provide students with the pseudocode to multiple recursive algorithms. Have students write the base case of the recursive methods and share it with a partner. The partner should then provide feedback, including any corrections or additions that may be needed.
12	4.16	**Look for a Pattern** Provide students with a recursive method and several different inputs. Have students run the recursive method, record the various outputs, and look for a pattern between the input and related output. Ask students to write one or two sentences as a broad description of what the recursive method is doing.
13	4.16	**Code Tracing** When looking at a recursive method to determine how many times it executes, have students create a call tree or a stack trace to show the method being called and the values of any parameters of each call. Students can then count up the number of times a statement executes or a method is called.

{ } After completing this unit, students will have covered all of the necessary content for the Data Set Lab, 2048 Lab, and Digit Recognition Lab, which can be found in **AP Classroom**.

SUGGESTED SKILLS

1.B

Determine what knowledge can be extracted from data.

5.A

Explain how computing impacts society, economy, and culture.

TOPIC 4.1
Ethical and Social Issues Around Data Collection

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
4.1.A Explain the risks to privacy from collecting and storing personal data on computer systems.	**4.1.A.1** When using a computer, personal privacy is at risk. When developing new programs, programmers should attempt to safeguard the personal privacy of the user.
4.1.B Explain the importance of recognizing data quality and potential issues when using a data set.	**4.1.B.1** *Algorithmic bias* describes systemic and repeated errors in a program that create unfair outcomes for a specific group of users. **4.1.B.2** Programmers should be aware of the data set collection method and the potential for bias when using this method before using the data to extrapolate new information or drawing conclusions. **4.1.B.3** Some data sets are incomplete or contain inaccurate data. Using such data in the development or use of a program can cause the program to work incorrectly or inefficiently.
4.1.C Identify an appropriate data set to use in order to solve a problem or answer a specific question.	**4.1.C.1** Contents of a data set might be related to a specific question or topic and might not be appropriate to give correct answers or extrapolate information for a different question or topic.

TOPIC 4.2
Introduction to Using Data Sets

SUGGESTED SKILLS

1.B

Determine what knowledge can be extracted from data.

Required Course Content

LEARNING OBJECTIVE

4.2.A

Represent patterns and algorithms that involve data sets found in everyday life using written language or diagrams.

ESSENTIAL KNOWLEDGE

4.2.A.1

A *data set* is a collection of specific pieces of information or data.

4.2.A.2

Data sets can be manipulated and analyzed to solve a problem or answer a question. When analyzing data sets, values within the set are accessed and utilized one at a time and then processed according to the desired outcome.

4.2.A.3

Data can be represented in a diagram by using a chart or table. This visual can be used to plan the algorithm that will be used to manipulate the data.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

TOPIC 4.3
Array Creation and Access

Required Course Content

LEARNING OBJECTIVE

4.3.A

Develop code used to represent collections of related data using one-dimensional (1D) array objects.

ESSENTIAL KNOWLEDGE

4.3.A.1

An *array* stores multiple values of the same type. The values can be either primitive values or object references.

4.3.A.2

The length of an array is established at the time of creation and cannot be changed. The length of an array can be accessed through the `length` attribute.

4.3.A.3

When an array is created using the keyword `new`, all of its elements are initialized to the default values for the element data type. The default value for `int` is `0`, for `double` is `0.0`, for `boolean` is `false`, and for a reference type is `null`.

4.3.A.4

Initializer lists can be used to create and initialize arrays.

4.3.A.5

Square brackets `[]` are used to access and modify an element in a 1D array using an index.

4.3.A.6

The valid index values for an array are `0` through one less than the length of the array, inclusive. Using an index value outside of this range will result in an `ArrayIndexOutOfBoundsException`.

TOPIC 4.4
Array Traversals

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

Required Course Content

LEARNING OBJECTIVE

4.4.A

Develop code used to traverse the elements in a 1D array and determine the result of these traversals.

ESSENTIAL KNOWLEDGE

4.4.A.1

Traversing an array is when repetition statements are used to access all or an ordered sequence of elements in an array.

4.4.A.2

Traversing an array with an indexed `for` loop or `while` loop requires elements to be accessed using their indices.

4.4.A.3

An enhanced `for` loop header includes a variable, referred to as the enhanced `for` loop variable. For each iteration of the enhanced `for` loop, the enhanced `for` loop variable is assigned a copy of an element without using its index.

4.4.A.4

Assigning a new value to the enhanced `for` loop variable does not change the value stored in the array.

4.4.A.5

When an array stores object references, the attributes can be modified by calling methods on the enhanced `for` loop variable. This does not change the object references stored in the array.

4.4.A.6

Code written using an enhanced `for` loop to traverse elements in an array can be rewritten using an indexed `for` loop or a `while` loop.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

TOPIC 4.5
Implementing Array Algorithms

Required Course Content

LEARNING OBJECTIVE

4.5.A

Develop code for standard and original algorithms for a particular context or specification that involves arrays and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE

4.5.A.1

There are standard algorithms that utilize array traversals to:

- determine a minimum or maximum value
- compute a sum or average
- determine if at least one element has a particular property
- determine if all elements have a particular property
- determine the number of elements having a particular property
- access all consecutive pairs of elements
- determine the presence or absence of duplicate elements
- shift or rotate elements left or right
- reverse the order of the elements

TOPIC 4.6
Using Text Files

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

3.C

Determine the result or output based on code that contains procedural abstractions.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

Required Course Content

LEARNING OBJECTIVE

4.6.A

Develop code to read data from a text file.

ESSENTIAL KNOWLEDGE

4.6.A.1

A *file* is storage for data that persists when the program is not running. The data in a file can be retrieved during program execution.

4.6.A.2

A file can be connected to the program using the `File` and `Scanner` classes.

4.6.A.3

A file can be opened by creating a `File` object, using the name of the file as the argument of the constructor.

- `File(String str)` is the `File` constructor that accepts a `String` file name to open for reading, where `str` is the pathname for the file.

4.6.A.4

When using the `File` class, it is required to indicate what to do if the file with the provided name cannot be opened. One way to accomplish this is to add `throws IOException` to the header of the method that uses the file. If the file name is invalid, the program will terminate.

4.6.A.5

The `File` and `IOException` classes are part of the `java.io` package. An `import` statement must be used to make these classes available for use in the program.

continued on next page

LEARNING OBJECTIVE

4.6.A

Develop code to read data from a text file.

ESSENTIAL KNOWLEDGE

4.6.A.6

The following `Scanner` methods and constructor—including what they do and when they are used—are part of the Java Quick Reference:

- `Scanner(File f)` is the `Scanner` constructor that accepts a `File` for reading.
- `int nextInt()` returns the next `int` read from the file or input source if available. If the next `int` does not exist or is out of range, it will result in an `InputMismatchException`.
- `double nextDouble()` returns the next `double` read from the file or input source. If the next `double` does not exist, it will result in an `InputMismatchException`.
- `boolean nextBoolean()` returns the next `boolean` read from the file or input source. If the next `boolean` does not exist, it will result in an `InputMismatchException`.
- `String nextLine()` returns the next line of text as a `String` read from the file or input source; can return the empty string if called immediately after another `Scanner` method that is reading from the file or input source.
- `String next()` returns the next `String` read from the file or input source.
- `boolean hasNext()` returns `true` if there is a next item to read in the file or input source; returns `false` otherwise.
- `void close()` closes this scanner.

EXCLUSION STATEMENT—*Accepting input from the keyboard is outside the scope of the AP Computer Science A course and exam.*

4.6.A.7

Using `nextLine` and the other `Scanner` methods together on the same input source sometimes requires code to adjust for the methods' different ways of handling whitespace.

EXCLUSION STATEMENT—*Writing or analyzing code that uses both* `nextLine` *and other* `Scanner` *methods on the same input source is outside the scope of the AP Computer Science A course and exam.*

continued on next page

LEARNING OBJECTIVE

4.6.A

Develop code to read data from a text file.

ESSENTIAL KNOWLEDGE

4.6.A.8

The following additional `String` method—including what it does and when it is used—is part of the Java Quick Reference:

- `String[] split(String del)` returns a `String` array where each element is a substring of `this String`, which has been split around matches of the given expression `del`.

EXCLUSION STATEMENT—*The parameter* `del` *uses a format called a regular expression. Writing or analyzing code that uses any of the special properties of regular expressions (e.g.,* `\\*`, `\\.`*) is outside the scope of the AP Computer Science A course and exam.*

4.6.A.9

A `while` loop can be used to detect if the file still contains elements to read by using the `hasNext` method as the condition of the loop.

4.6.A.10

A file should be closed when the program is finished using it. The `close` method from `Scanner` is called to close the file.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

TOPIC 4.7
Wrapper Classes

Required Course Content

LEARNING OBJECTIVE

4.7.A

Develop code to use `Integer` and `Double` objects from their primitive counterparts and determine the result of using these objects.

ESSENTIAL KNOWLEDGE

4.7.A.1

The `Integer` class and `Double` class are part of the `java.lang` package. An `Integer` object is immutable, meaning once an `Integer` object is created, its attributes cannot be changed. A `Double` object is immutable, meaning once a `Double` object is created, its attributes cannot be changed.

4.7.A.2

Autoboxing is the automatic conversion that the Java compiler makes between primitive types and their corresponding object wrapper classes. This includes converting an `int` to an `Integer` and a `double` to a `Double`. The Java compiler applies autoboxing when a primitive value is:

- passed as a parameter to a method that expects an object of the corresponding wrapper class
- assigned to a variable of the corresponding wrapper class

4.7.A.3

Unboxing is the automatic conversion that the Java compiler makes from the wrapper class to the primitive type. This includes converting an `Integer` to an `int` and a `Double` to a `double`. The Java compiler applies unboxing when a wrapper class object is:

- passed as a parameter to a method that expects a value of the corresponding primitive type
- assigned to a variable of the corresponding primitive type

continued on next page

LEARNING OBJECTIVE

4.7.A

Develop code to use `Integer` and `Double` objects from their primitive counterparts and determine the result of using these objects.

ESSENTIAL KNOWLEDGE

4.7.A.4

The following class `Integer` method—including what it does and when it is used—is part of the Java Quick Reference:

- `static int parseInt(String s)` returns the `String` argument as an `int`.

4.7.A.5

The following class `Double` method—including what it does and when it is used—is part of the Java Quick Reference:

- `static double parseDouble(String s)` returns the `String` argument as a `double`.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

TOPIC 4.8
`ArrayList` Methods

Required Course Content

LEARNING OBJECTIVE

4.8.A

Develop code for collections of related objects using `ArrayList` objects and determine the result of calling methods on these objects.

ESSENTIAL KNOWLEDGE

4.8.A.1

An `ArrayList` object is mutable in size and contains object references.

4.8.A.2

The `ArrayList` constructor `ArrayList()` constructs an empty list.

4.8.A.3

Java allows the generic type `ArrayList<E>`, where the type parameter `E` specifies the type of the elements. When `ArrayList<E>` is specified, the types of the reference parameters and return type when using the `ArrayList` methods are type `E`. `ArrayList<E>` is preferred over `ArrayList`. For example, `ArrayList<String> names = new ArrayList<String>();` allows the compiler to find errors that would otherwise be found at run-time.

4.8.A.4

The `ArrayList` class is part of the `java.util` package. An `import` statement must be used to make this class available for use in the program.

continued on next page

LEARNING OBJECTIVE

4.8.A

Develop code for collections of related objects using `ArrayList` objects and determine the result of calling methods on these objects.

ESSENTIAL KNOWLEDGE

4.8.A.5

The following `ArrayList` methods—including what they do and when they are used—are part of the Java Quick Reference:

- `int size()` returns the number of elements in the list.
- `boolean add(E obj)` appends `obj` to end of list; returns `true`.
- `void add(int index, E obj)` inserts `obj` at position `index` (`0 <= index <= size`), moving elements at position `index` and higher to the right (adds 1 to their indices) and adds 1 to size.
- `E get(int index)` returns the element at position `index` in the list.
- `E set(int index, E obj)` replaces the element at position `index` with `obj`; returns the element formerly at position `index`.
- `E remove(int index)` removes element from position `index`, moving elements at position `index + 1` and higher to the left (subtracts 1 from their indices) and subtracts 1 from size; returns the element formerly at position `index`.

4.8.A.6

The indices for an `ArrayList` start at `0` and end at the number of elements `- 1`.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

TOPIC 4.9
ArrayList Traversals

Required Course Content

LEARNING OBJECTIVE

4.9.A

Develop code used to traverse the elements of an `ArrayList` and determine the results of these traversals.

ESSENTIAL KNOWLEDGE

4.9.A.1

Traversing an `ArrayList` is when iteration or recursive statements are used to access all or an ordered sequence of the elements in an `ArrayList`.

4.9.A.2

Deleting elements during a traversal of an `ArrayList` requires the use of special techniques to avoid skipping elements.

4.9.A.3

Attempting to access an index value outside of its range will result in an `IndexOutOfBoundsException`.

4.9.A.4

Changing the size of an `ArrayList` while traversing it using an enhanced `for` loop can result in a `ConcurrentModificationException`. Therefore, when using an enhanced `for` loop to traverse an `ArrayList`, you should not add or remove elements.

TOPIC 4.10
Implementing `ArrayList` Algorithms

Required Course Content

LEARNING OBJECTIVE

4.10.A

Develop code for standard and original algorithms for a particular context or specification that involve `ArrayList` objects and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE

4.10.A.1

There are standard `ArrayList` algorithms that utilize traversals to:

- determine a minimum or maximum value
- compute a sum or average
- determine if at least one element has a particular property
- determine if all elements have a particular property
- determine the number of elements having a particular property
- access all consecutive pairs of elements
- determine the presence or absence of duplicate elements
- shift or rotate elements left or right
- reverse the order of the elements
- insert elements
- delete elements

4.10.A.2

Some algorithms require multiple `String`, array, or `ArrayList` objects to be traversed simultaneously.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

TOPIC 4.11
2D Array Creation and Access

Required Course Content

LEARNING OBJECTIVE

4.11.A

Develop code used to represent collections of related data using two-dimensional (2D) array objects.

ESSENTIAL KNOWLEDGE

4.11.A.1

A 2D array is stored as an array of arrays. Therefore, the way 2D arrays are created and indexed is similar to 1D array objects. The size of a 2D array is established at the time of creation and cannot be changed. 2D arrays can store either primitive data or object reference data.

EXCLUSION STATEMENT—*Nonrectangular 2D array objects are outside the scope of the AP Computer Science A course and exam.*

4.11.A.2

When a 2D array is created using the keyword `new`, all of its elements are initialized to the default values for the element data type. The default value for `int` is `0`, for `double` is `0.0`, for `boolean` is `false`, and for a reference type is `null`.

4.11.A.3

The initializer list used to create and initialize a 2D array consists of initializer lists that represent 1D arrays; for example, `int[][] arr2D = { {1, 2, 3}, {4, 5, 6} };`.

4.11.A.4

The square brackets `[row][col]` are used to access and modify an element in a 2D array. For the purposes of the exam, when accessing the element at `arr[first][second]`, the first index is used for rows, the second index is used for columns.

continued on next page

LEARNING OBJECTIVE

4.11.A

Develop code used to represent collections of related data using two-dimensional (2D) array objects.

ESSENTIAL KNOWLEDGE

4.11.A.5

A single array that is a row of a 2D array can be accessed using the 2D array name and a single set of square brackets containing the row index.

4.11.A.6

The number of rows contained in a 2D array can be accessed through the `length` attribute. The valid row index values for a 2D array are `0` through one less than the number of rows or the length of the array, inclusive. The number of columns contained in a 2D array can be accessed through the `length` attribute of one of the rows. The valid column index values for a 2D array are `0` through one less than the number of columns or the length of any given row of the array, inclusive. For example, given a 2D array named `values`, the number of rows is `values.length` and the number of columns is `values[0].length`. Using an index value outside of these ranges will result in an `ArrayIndexOutOfBoundsException`.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

TOPIC 4.12
2D Array Traversals

Required Course Content

LEARNING OBJECTIVE

4.12.A

Develop code used to traverse the elements in a 2D array and determine the result of these traversals.

ESSENTIAL KNOWLEDGE

4.12.A.1

Nested iteration statements are used to traverse and access all or an ordered sequence of elements in a 2D array. Since 2D arrays are stored as arrays of arrays, the way 2D arrays are traversed using `for` loops and enhanced `for` loops is similar to 1D array objects. Nested iteration statements can be written to traverse the 2D array in row-major order, column-major order, or a uniquely defined order. *Row-major order* refers to an ordering of 2D array elements where traversal occurs across each row, whereas *column-major order* traversal occurs down each column.

4.12.A.2

The outer loop of a nested enhanced `for` loop used to traverse a 2D array traverses the rows. Therefore, the enhanced `for` loop variable must be the type of each row, which is a 1D array. The inner loop traverses a single row. Therefore, the inner enhanced `for` loop variable must be the same type as the elements stored in the 1D array. Assigning a new value to the enhanced `for` loop variable does not change the value stored in the array.

TOPIC 4.13
Implementing 2D Array Algorithms

Required Course Content

LEARNING OBJECTIVE

4.13.A

Develop code for standard and original algorithms for a particular context or specification that involves 2D arrays and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE

4.13.A.1

There are standard algorithms that utilize 2D array traversals to:

- determine a minimum or maximum value of all the elements or for a designated row, column, or other subsection
- compute a sum or average of all the elements or for a designated row, column, or other subsection
- determine if at least one element has a particular property in the entire 2D array or for a designated row, column, or other subsection
- determine if all elements of the 2D array or a designated row, column, or other subsection have a particular property
- determine the number of elements in the 2D array or in a designated row, column, or other subsection having a particular property
- access all consecutive pairs of elements
- determine the presence or absence of duplicate elements in the 2D array or in a designated row, column, or other subsection
- shift or rotate elements in a row left or right or in a column up or down
- reverse the order of the elements in a row or column

SUGGESTED SKILLS

2B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

4.A

Describe the behavior of a code segment or program.

TOPIC 4.14
Searching Algorithms

Required Course Content

LEARNING OBJECTIVE

4.14.A

Develop code used for linear search algorithms to search for specific information in a collection and determine the results of executing a search.

ESSENTIAL KNOWLEDGE

4.14.A.1

Linear search algorithms are standard algorithms that check each element in order until the desired value is found or all elements in the array or `ArrayList` have been checked. Linear search algorithms can begin the search process from either end of the array or `ArrayList`.

4.14.A.2

When applying linear search algorithms to 2D arrays, each row must be accessed then linear search applied to each row of the 2D array.

TOPIC 4.15
Sorting Algorithms

SUGGESTED SKILLS

3.B

Determine the result or output based on code that contains data abstractions.

4.A

Describe the behavior of a code segment or program.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

Required Course Content

LEARNING OBJECTIVE

4.15.A

Determine the result of executing each step of sorting algorithms to sort the elements of a collection.

ESSENTIAL KNOWLEDGE

4.15.A.1

Selection sort and insertion sort are iterative sorting algorithms that can be used to sort elements in an array or `ArrayList`.

4.15.A.2

Selection sort repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it into its correct (and final) position in the sorted portion of the list.

4.15.A.3

Insertion sort inserts an element from the unsorted portion of a list into its correct (but not necessarily final) position in the sorted portion of the list by shifting elements of the sorted portion to make room for the new element.

SUGGESTED SKILLS

3.C

Determine the result or output based on code that contains procedural abstractions.

4.A

Describe the behavior of a code segment or program.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

TOPIC 4.16
Recursion

Required Course Content

LEARNING OBJECTIVE

4.16.A

Determine the result of calling recursive methods.

ESSENTIAL KNOWLEDGE

4.16.A.1

A *recursive method* is a method that calls itself. Recursive methods contain at least one base case, which halts the recursion, and at least one recursive call. Recursion is another form of repetition.

4.16.A.2

Each recursive call has its own set of local variables, including the parameters. Parameter values capture the progress of a recursive process, much like loop control variable values capture the progress of a loop.

4.16.A.3

Any recursive solution can be replicated through the use of an iterative approach and vice versa.

EXCLUSION STATEMENT—*Writing recursive code is outside the scope of the AP Computer Science A course and exam.*

TOPIC 4.17
Recursive Searching and Sorting

SUGGESTED SKILLS

4.A

Describe the behavior of a code segment or program.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

Required Course Content

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
4.17.A Determine the result of executing recursive algorithms that use strings or collections.	**4.17.A.1** Recursion can be used to traverse `String` objects, arrays, and `ArrayList` objects.
4.17.B Determine the result of each iteration of a binary search algorithm used to search for information in a collection.	**4.17.B.1** Data must be in sorted order to use the binary search algorithm. *Binary search* starts at the middle of a sorted array or `ArrayList` and eliminates half of the array or `ArrayList` in each recursive call until the desired value is found or all elements have been eliminated. **4.17.B.2** Binary search is typically more efficient than linear search. **EXCLUSION STATEMENT**—*Search algorithms other than linear and binary search are outside the scope of the AP Computer Science A course and exam.* **4.17.B.3** The binary search algorithm can be written either iteratively or recursively.
4.17.C Determine the result of each iteration of the merge sort algorithm when used to sort a collection.	**4.17.C.1** *Merge sort* is a recursive sorting algorithm that can be used to sort elements in an array or `ArrayList`. **EXCLUSION STATEMENT**—*Sorting algorithms other than selection, insertion, and merge sort are outside the scope of the AP Computer Science A course and exam.*

continued on next page

LEARNING OBJECTIVE	ESSENTIAL KNOWLEDGE
4.17.C Determine the result of each iteration of the merge sort algorithm when used to sort a collection.	**4.17.C.2** Merge sort repeatedly divides an array into smaller subarrays until each subarray is one element and then recursively merges the sorted subarrays back together in sorted order to form the final sorted array.

AP COMPUTER SCIENCE A

Instructional Approaches


elloWorld.ja
*/
public class He
{
pu
ain